

datalex



# Cloud Computing



## Alexandre BERGERE

Head of Data & AI Engineer & Partners at DataGalaxy, Data Architect independent
Delta & openLineage lover

**Data Expert**
(Audit, Modeling, Urbanization, BI's project – Azure, Big Data, Spark, openLineage & Delta lover)

**Trainer**
(BI, Big Data, MongoDB, NoSQL, SQL, Cloud Computing)

**Speaker**
(Pass Summit 2018, Spark AI Tour 2020, …)

**Cloud & data tech articles**
(medium, LinkedIn, slideshare)

### Avanade
Sr Anls, Data Analyst

2016 - 2019

### Trainer
2016 - x

ESAIP, ESME, ECS, NEOMA, ESEO, EFREI, Sorbonne Paris

*BI, Big Data, MongoDB, NoSQL, Panorama de la data, Cloud computing*

### OpenClassroom
Mentor

2019 - x

### Datalex
2019 - x

Data Architect Freelance

### DataRedKite
CTO & Co-Founder

2020 - 2023

### DataGalaxy
2021 - x

Head of Data & AI Engineer

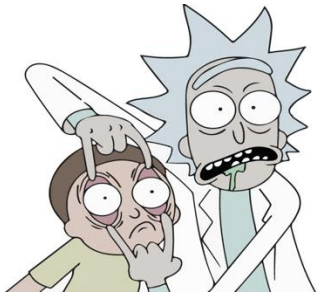
# Don’t get lost




Practice



Demo

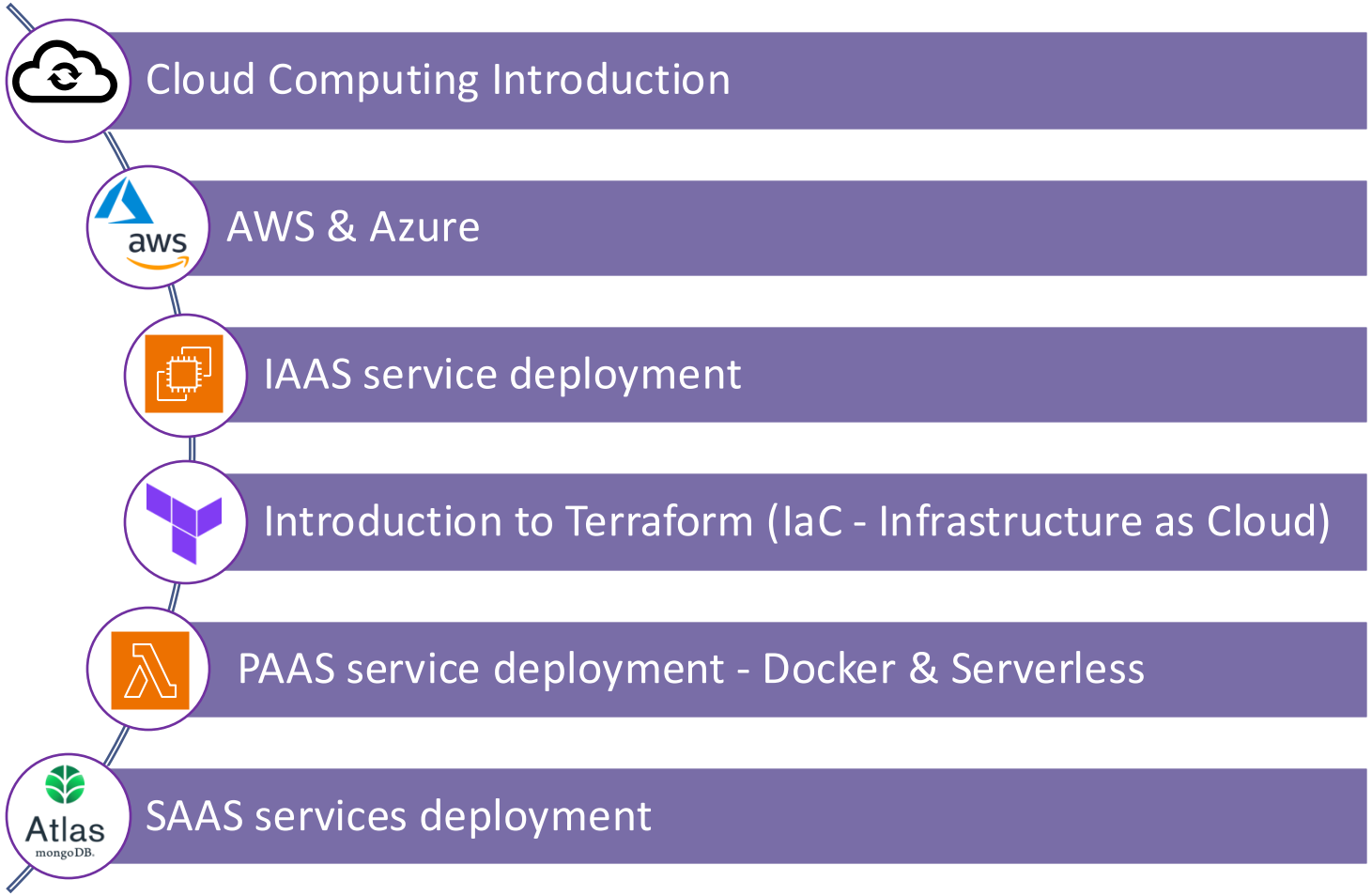


Important



Tips

# Summary


Cloud Computing Introduction
aws
AWS & Azure
IAAS service deployment
Introduction to Terraform (IaC - Infrastructure as Cloud)
PAAS service deployment - Docker & Serverless
Atlas
mongoDB.
SAAS services deployment

# AWS

## Basics concepts



# AWS - The architectural need

It’s 2000, and Amazon.com’s new shopping website service is struggling to become highly available and scale efficiently.

Amazon.com’s e-commerce tools were “a jumbled mess:”

- Applications and architectures were built without proper planning
- Services had to be separated from each other

**Solution:** Tools became a set of well-documented APIs, which became the standard for service development at Amazon.

https://techcrunch.com/2016/07/02/andy-jassys-brief-history-of-the-genesis-of-aws/

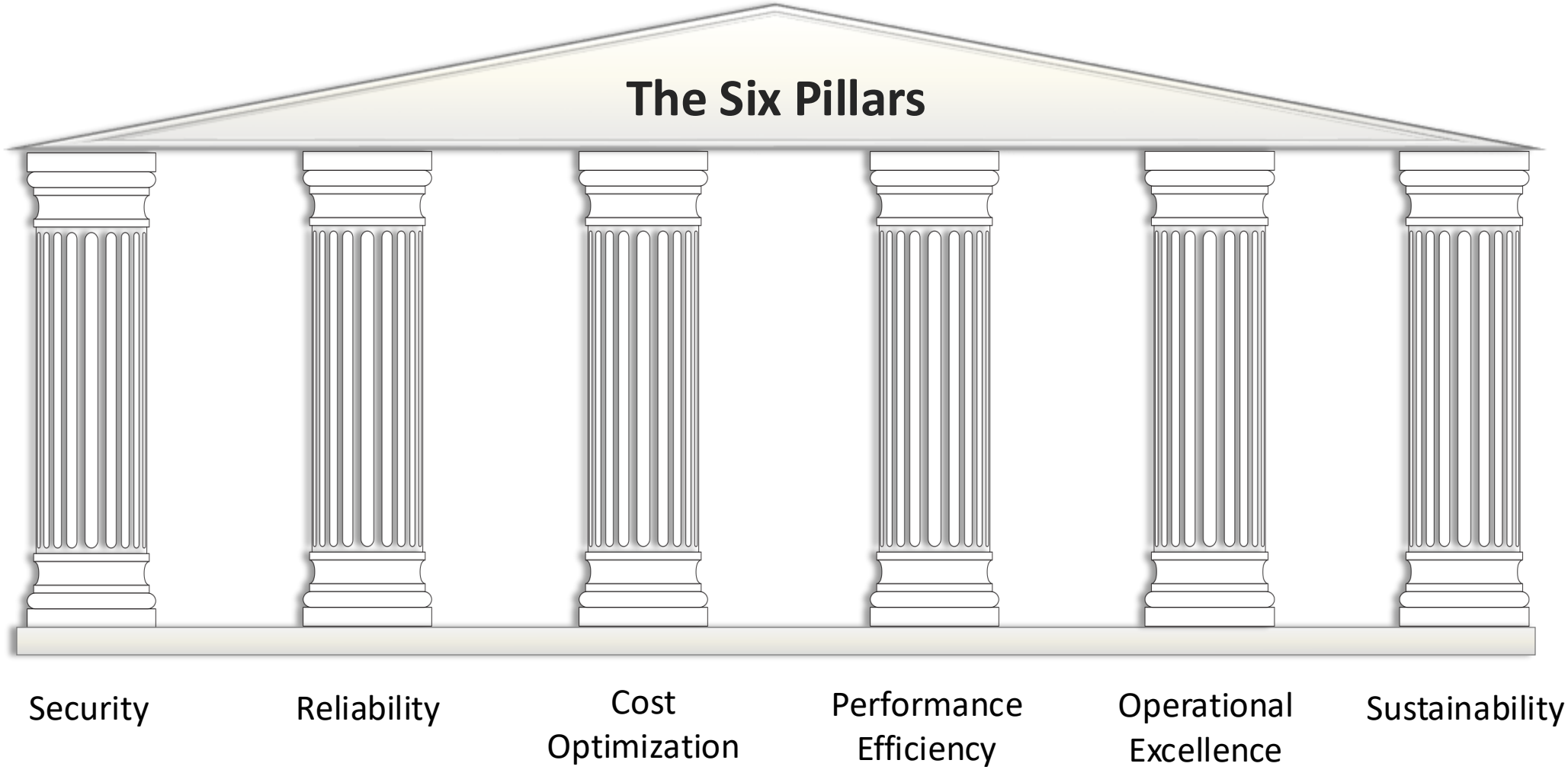
# AWS - Problems Persisted

Amazon.com still struggled to build applications quickly.

- Database, compute, and storage components took 3 months to build.
- Each team built their own resources, with no planning for scale or re-usability.

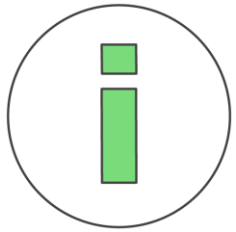
**Solution:** Built internal services to create highly available, scalable, and reliable architectures on top of their infrastructure. In 2006, started selling these services as AWS.

# The AWS Well-Architected Framework



# The Six Pillars

## Security



Identity foundation



Enabling traceability



Security at all layers



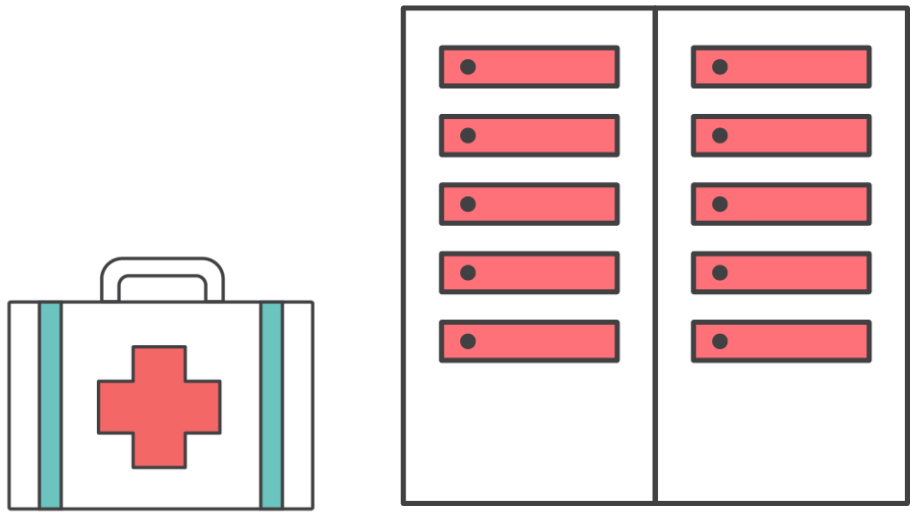
Risk assessment and mitigation strategies

https://d1.awsstatic.com/whitepapers/architecture/AWS-Security-Pillar.pdf

# The Six Pillars

## Reliability

- Dynamically acquire computing resources to meet demand
- Recover quickly from infrastructure or service failures
- Mitigate disruptions such as:
  - Misconfigurations
  - Transient network issues

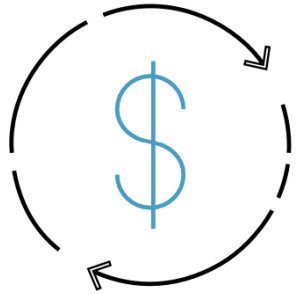


https://d1.awsstatic.com/whitepapers/architecture/AWS-Reliability-Pillar.pdf

# The Six Pillars

## Cost Optimization

- Measure efficiency
- Eliminate unneeded expense
- Consider using managed services



https://d1.awsstatic.com/whitepapers/architecture/AWS-Cost-Optimization-Pillar.pdf

# The Six Pillars

## Operational Excellence

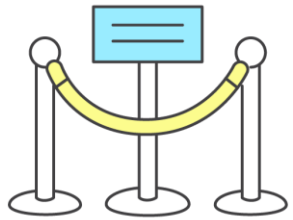
- The ability to run and monitor systems
- To continually improve supporting process and procedures



Deployed



Updated



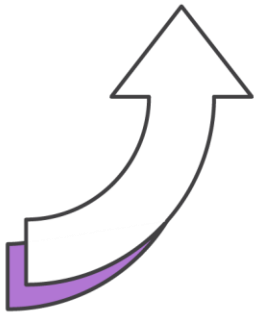
Operated

https://d1.awsstatic.com/whitepapers/architecture/AWS-Operational-Excellence-Pillar.pdf

# The Six Pillars

## Performance Efficiency

- Choose efficient resources and maintain their efficiency as demand changes
- Democratize advanced technologies
- Mechanical sympathy

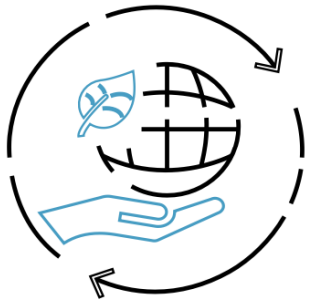


https://d1.awsstatic.com/whitepapers/architecture/AWS-Performance-Efficiency-Pillar.pdf

# The Six Pillars

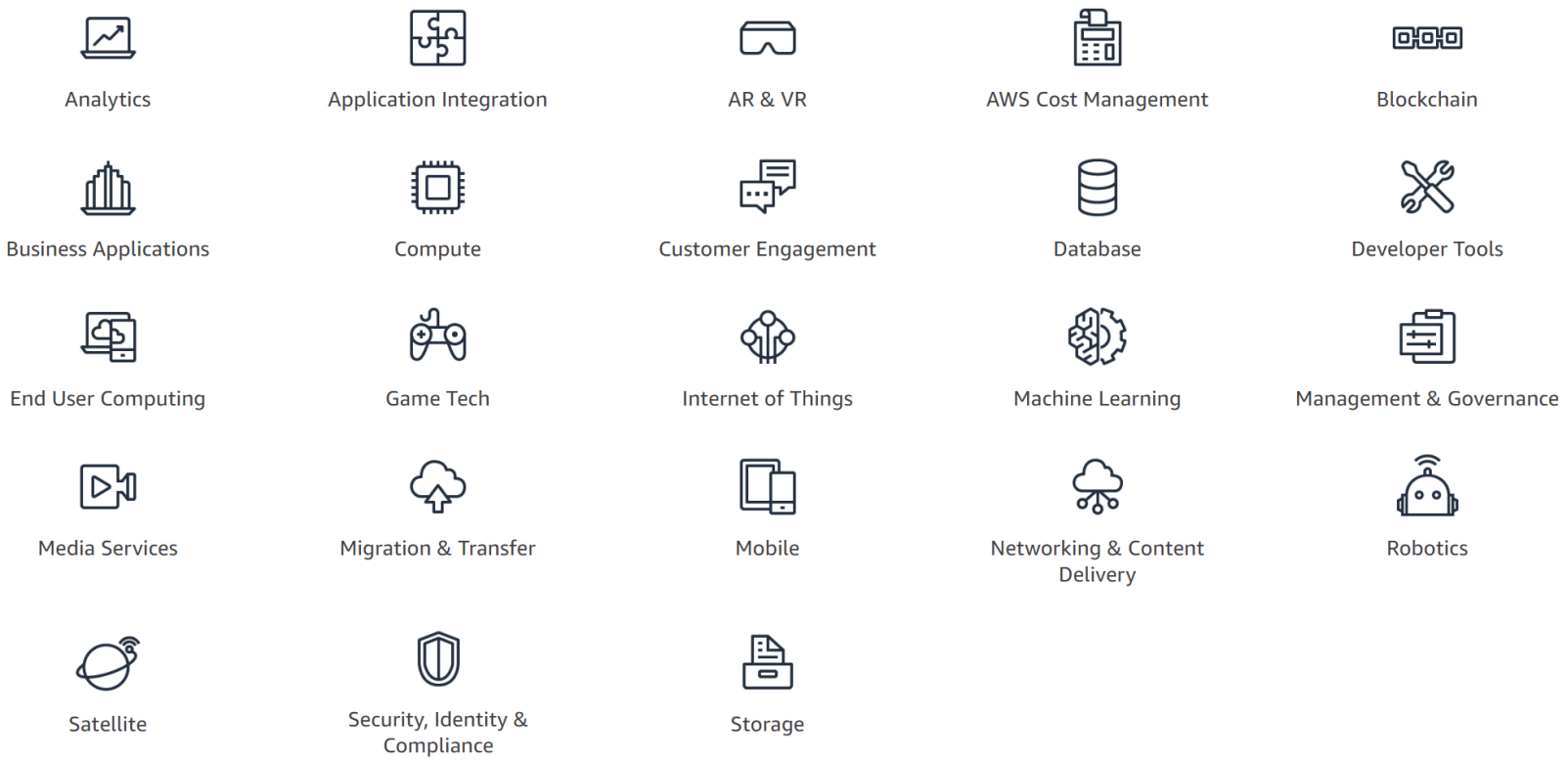
## Sustainability

- Understand your impact
- Establish sustainability goals
- Maximize utilization
- Anticipate and adopt new, more efficient hardware and software offerings
- Use managed services
- Reduce the downstream impact of your cloud workloads

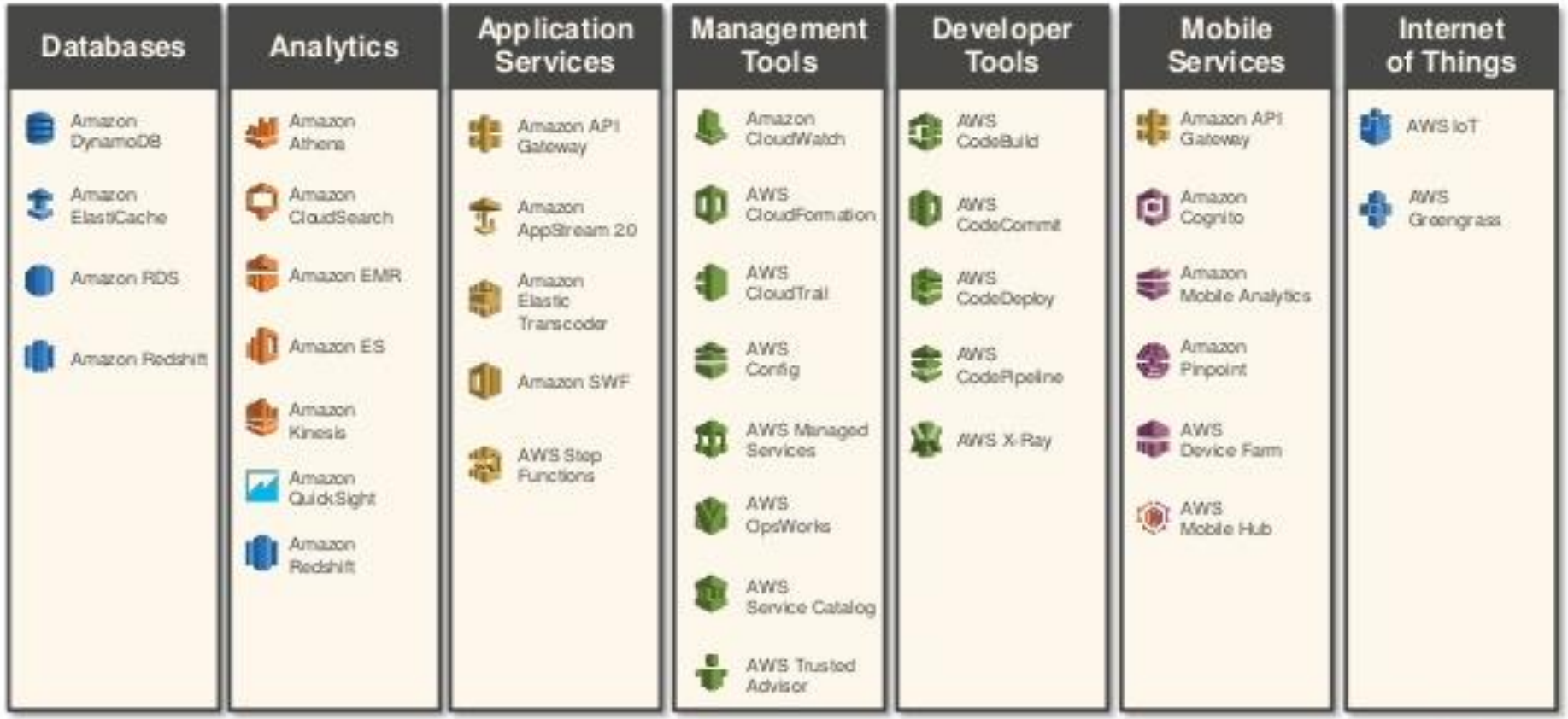


https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html

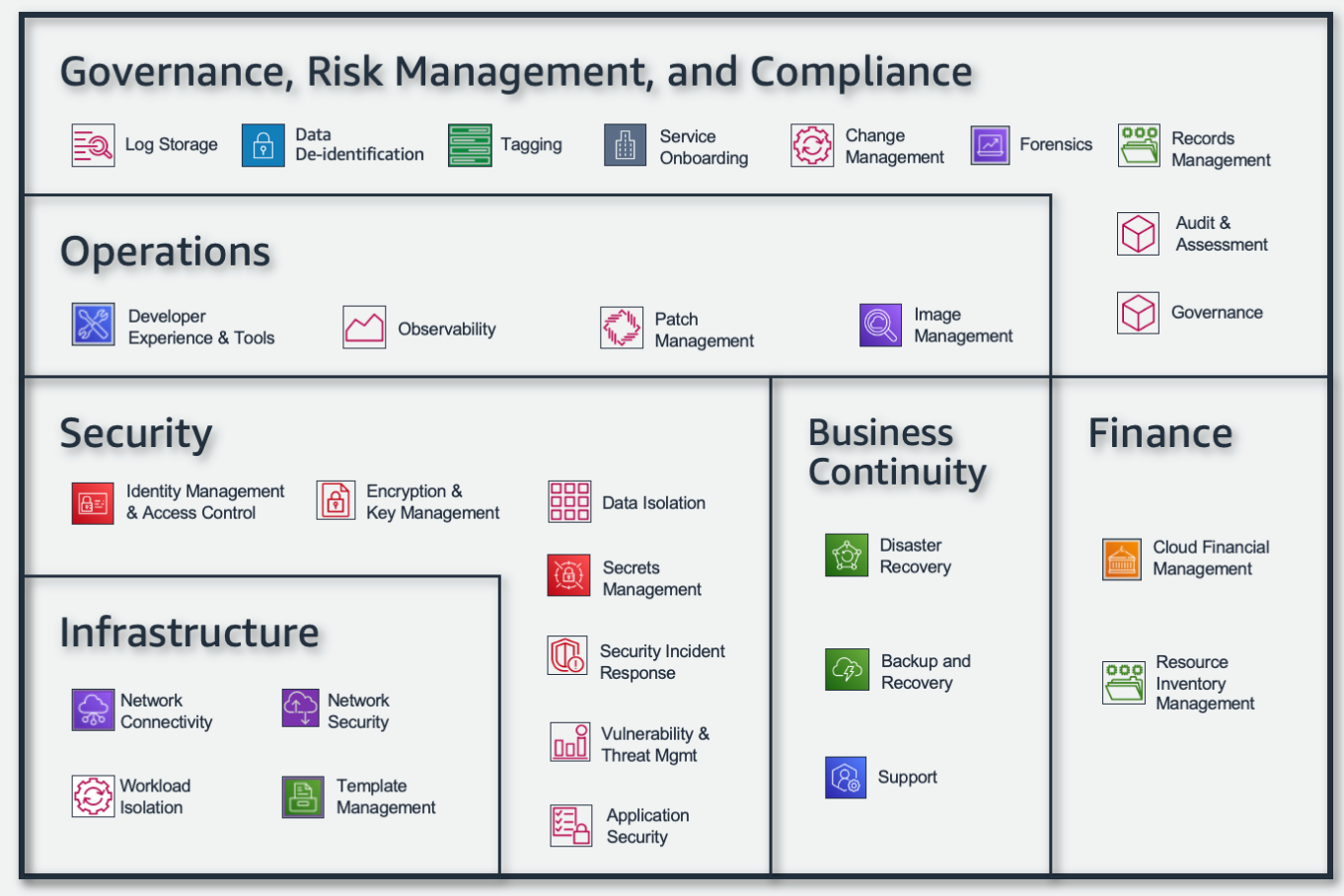
# AWS Services



# AWS Foundation Services



# AWS Cloud Foundation




https://aws.amazon.com/architecture/cloud-foundations/?nc1=h_ls

# Compute



Amazon EC2
(Amazon Elastic Compute Cloud)

web service that provides secure, resizable compute capacity in the cloud. It is designed to make web-scale cloud computing easier for developers.



AWS Lambda

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume - there is no charge when your code is not running.

# Storage



Amazon Block Storage
(Amazon EBS)

- A block storage
- You need an EC2 instance to attach it to



Amazon Simple Storage Services
(Amazon S3)

- An object store
- Great for storing logs



Amazon S3 Glacier

- Long term archive storage
- Extremely cheap to store
- Potentially very expensive to retrieve
- Takes up to 4 hours to "read back" your data



Amazon Elastic File System
(Amazon EFS)

- Network filesystem
- expensive compared to EBS

# Network - Security

## CDN



Amazon CloudFront

Amazon CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency, high transfer speeds, all within a developer-friendly environment.

## DNS



Amazon Route 53

Amazon Route 53 is a highly available and scalable cloud Domain Name System (DNS) web service.

## Firewall



AWS WAF

AWS WAF is a web application firewall that helps protect your web applications from common web exploits that could affect application availability, compromise security, or consume excessive resources.

## DDOS



AWS Shield

AWS Shield is a managed Distributed Denial of Service (DDoS) protection service that safeguards applications running on AWS.

# Data storage

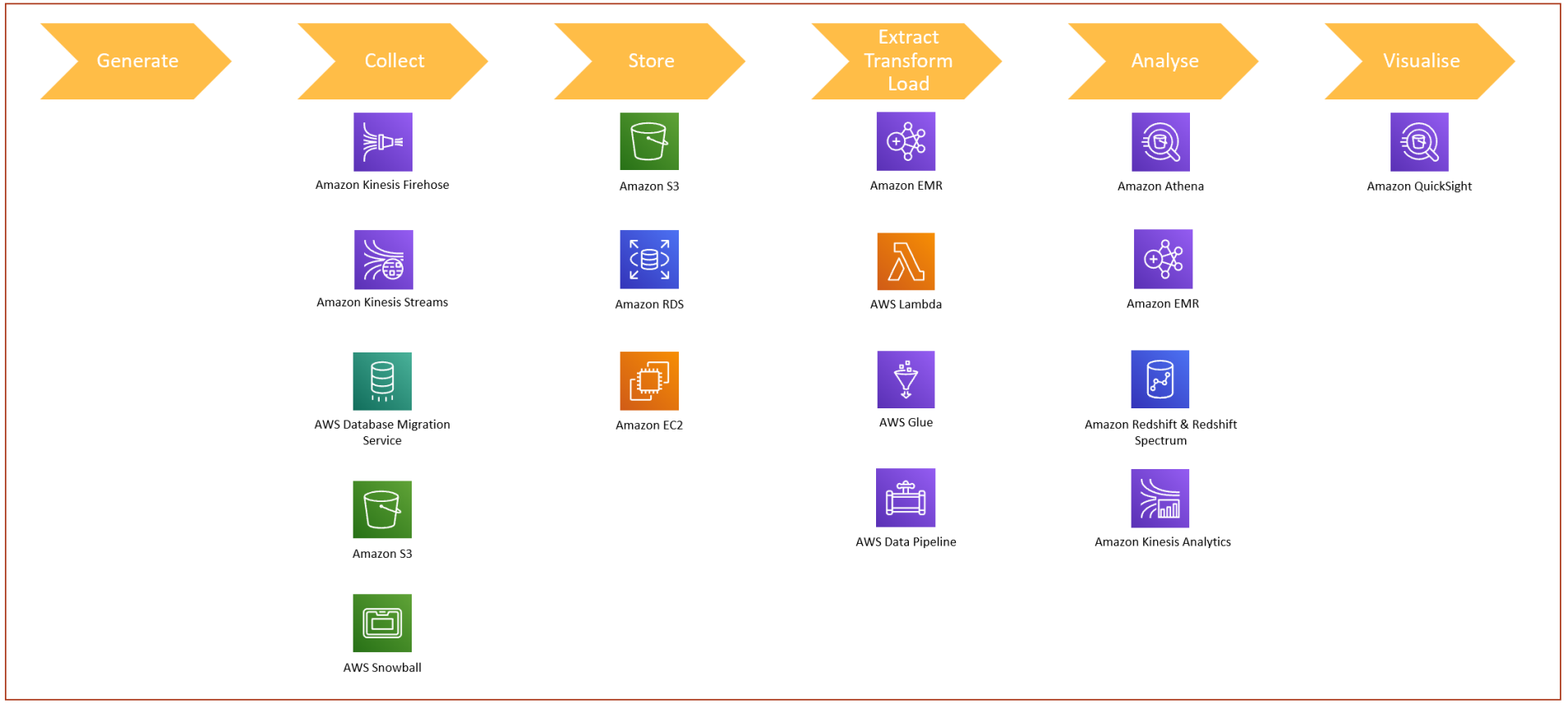
| Relational | Key-value Document | Document | In Memory | Graph | Search | Time-series | Ledger |
|---|---|---|---|---|---|---|---|
|  Amazon RDS |  Amazon DynamoDB |  Amazon DocumentDB |  Amazon ElastiCache |  Amazon Neptune |  Amazon Elasticsearch Service |  Amazon Timestream |  Amazon Quantum Ledger Database |
|  Amazon Aurora | | | | | | | |

# Analytics

| Datawarehouse | Data discovery | Data Analytics |
| --- | --- | --- |
|  Amazon Redshift |  Amazon Athena |  Amazon EMR |

| Stream |
| --- |
|  Amazon Kinesis |

# Analytics


Generate
Collect
Amazon Kinesis Firehose
Amazon Kinesis Streams
AWS Database Migration Service
Amazon S3
AWS Snowball
Store
Amazon S3
Amazon RDS
Amazon EC2
Extract Transform Load
Amazon EMR
AWS Lambda
AWS Glue
AWS Data Pipeline
Analyse
Amazon Athena
Amazon EMR
Amazon Redshift & Redshift Spectrum
Amazon Kinesis Analytics
Visualise
Amazon QuickSight

# Management Tool

| Firewall | Monitoring |
| --- | --- |
|  |  |
| AWS CloudFormation | Amazon CloudWatch |
| AWS CloudFormation provides a common language for you to describe and provision all the infrastructure resources in your cloud environment. | Amazon CloudWatch is a monitoring and management service built for developers, system operators, site reliability engineers (SRE), and IT managers. |

# Web

| APIs at any scale | Authentification | Build mobile and web apps that scale | Microservice |
| --- | --- | --- | --- |
|  |  |  |  |
| Amazon API Gateway | Amazon Cognito | AWS Amplify | Amazon SQS<br>(Simple Queue Service) |
| Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. | Amazon Cognito lets you add user sign-up, sign-in, and access control to your web and mobile apps quickly and easily. | AWS Amplify makes it easy to create, configure, and implement scalable mobile and web apps powered by AWS. | Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. |

# AWS regions

US East (N. Virginia) us-east-1
US East (Ohio) us-east-2
US West (N. California) us-west-1
US West (Oregon) us-west-2

Africa (Cape Town) af-south-1

Asia Pacific (Hong Kong) ap-east-1
Asia Pacific (Mumbai) ap-south-1
Asia Pacific (Osaka) ap-northeast-3
Asia Pacific (Seoul) ap-northeast-2
Asia Pacific (Singapore) ap-southeast-1
Asia Pacific (Sydney) ap-southeast-2
Asia Pacific (Tokyo) ap-northeast-1

Canada (Central) ca-central-1

Europe (Frankfurt) eu-central-1
Europe (Ireland) eu-west-1
Europe (London) eu-west-2
Europe (Milan) eu-south-1
Europe (Paris) eu-west-3
Europe (Stockholm) eu-north-1

Middle East (Bahrain) me-south-1

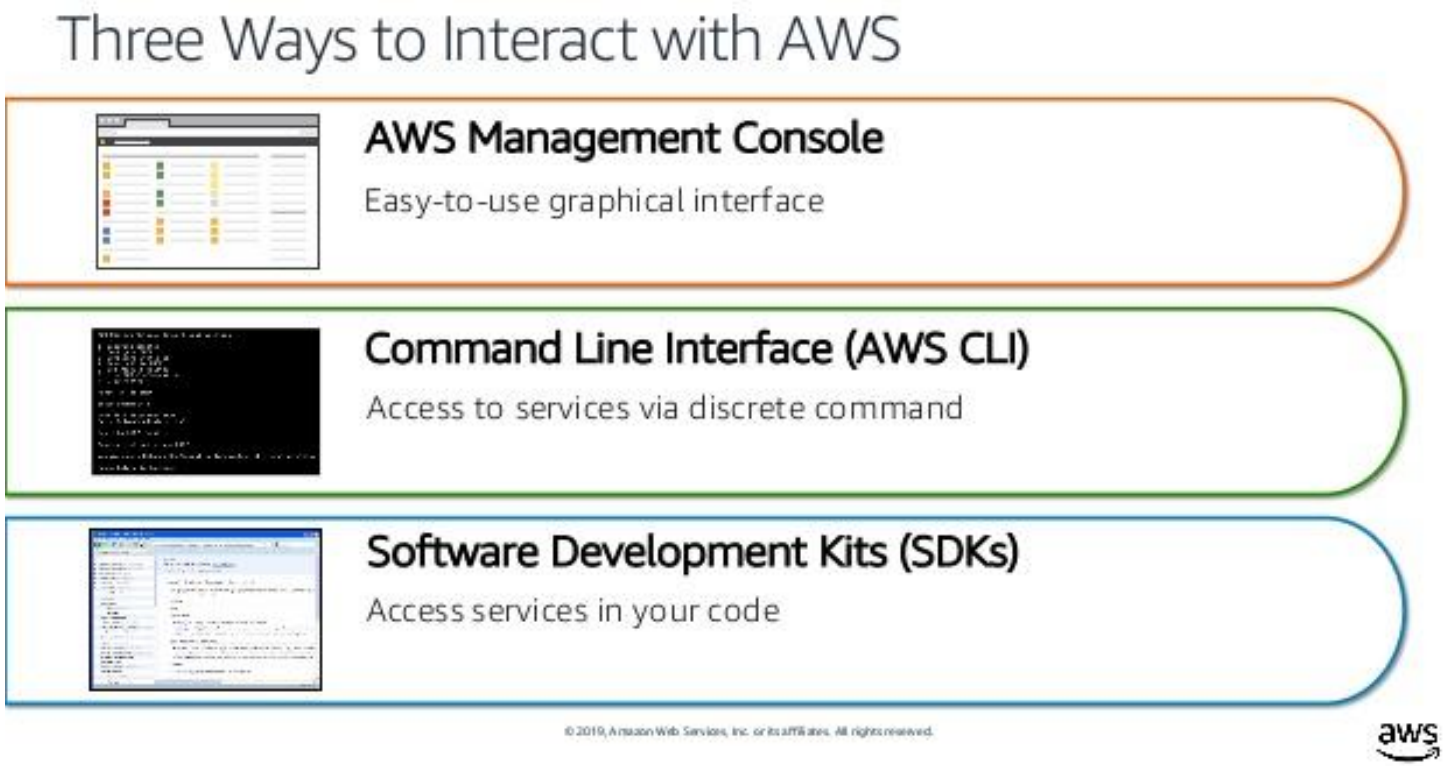
South America (São Paulo) sa-east-1

# AWS

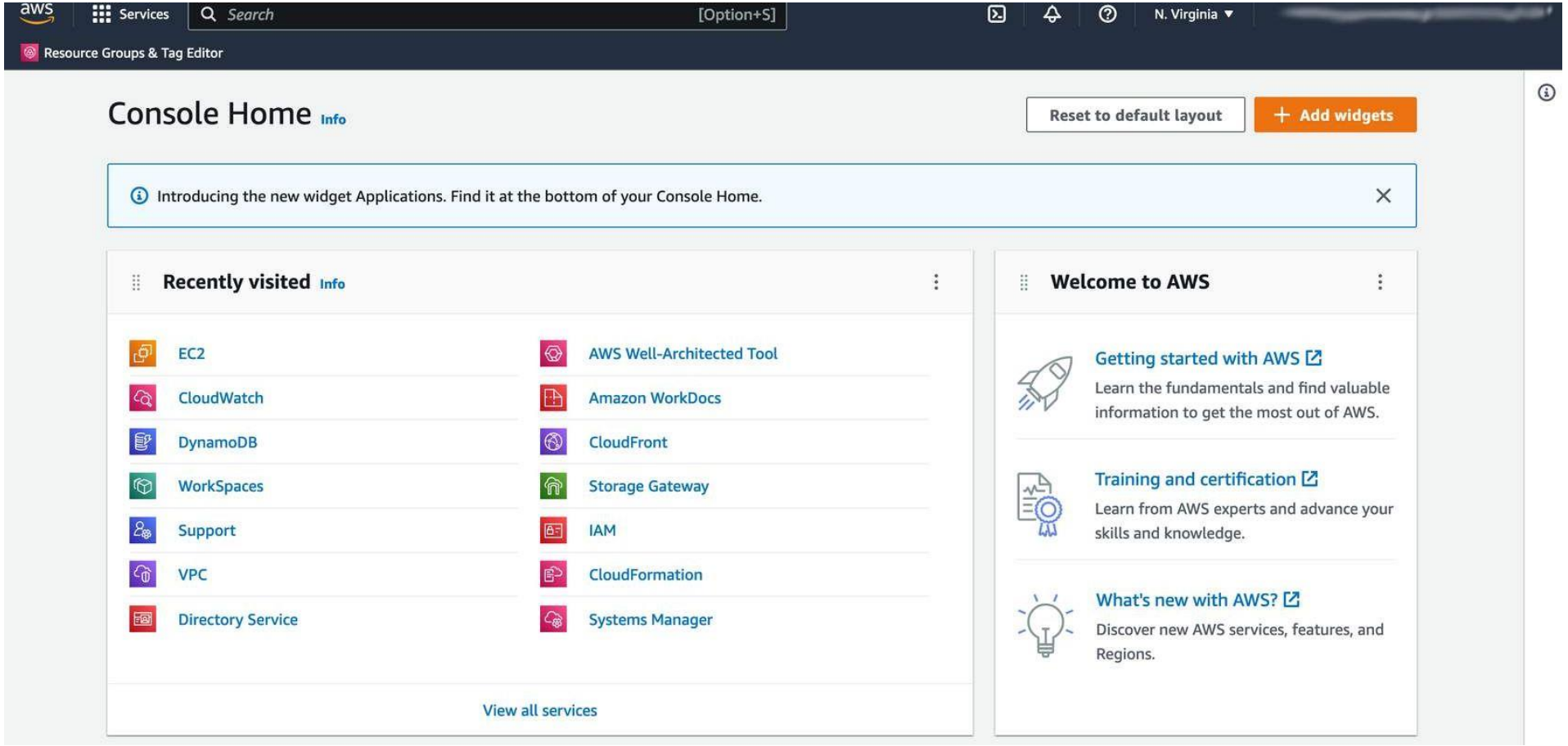
## Interact with AWS



# Interact with AWS



# AWS Management Console




http://console.aws.amazon.com/

# AWS Portal



# AWS root user

When you first create an AWS account, you begin with a single sign-in identity that has complete access to all AWS services and resources in the account. This identity is called the AWS root user and is accessed by signing in with the email address and password that were used to create the account.

**AWS root user credentials**

The AWS root user has two sets of credentials associated with it. One set of credentials is the email address and password that were used to create the account. This allows you to access the AWS Management Console. The second set of credentials is called access keys, which allow you to make programmatic requests from the AWS Command Line Interface (AWS CLI) or AWS API.

Access keys consist of two parts:

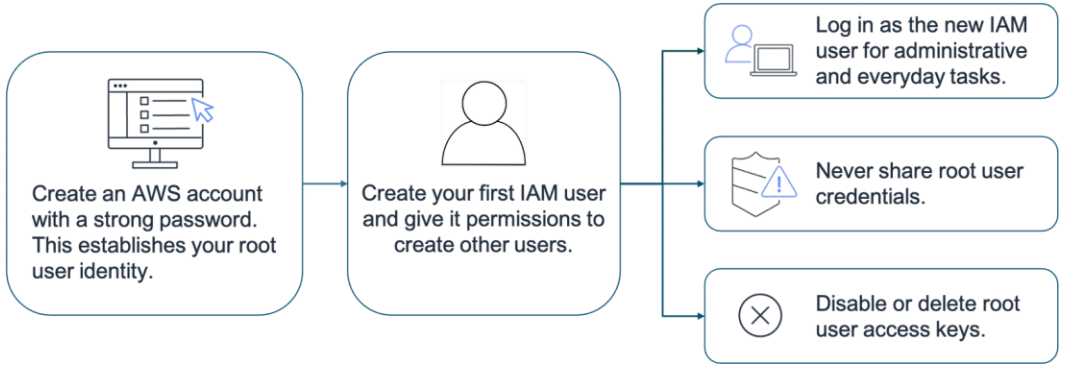
- **Access key ID:** for example, ***A2lAl5EXAMPLE***
- **Secret access key:** for example, ***wJalrFE/KbEKxE***

# AWS root user best practices

The root user has complete access to all AWS services and resources in your account, including your billing and personal information. Therefore, you should securely lock away the credentials associated with the root user and not use the root user for everyday tasks.

To ensure the safety of the root user, follow these best practices:

- Choose a strong password for the root user.
- Enable multi-factor authentication (MFA) for the root user.
- Never share your root user password or access keys with anyone.
- Disable or delete the access keys associated with the root user.
- Create an Identity and Access Management (IAM) user for administrative tasks or everyday tasks.



# LAB – AWS Management Console




1. Create an AWS Account.
2. Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy
3. Create access key & download the credentials csv file
4. Explore services.

# AWS account



Your turn !

## AWS Free Tier

Gain free, hands-on experience with the AWS platform, products, and services

Create a Free Account

### Types of offers

Explore more than 60 products and start building on AWS using the free tier. Three different types of free offers are available depending on the product used. See below for details on each product.

**Always free**

These free tier offers do not expire and are available to all AWS customers

**12 months free**

Enjoy these offers for 12-months following your initial sign-up date to AWS

**Trials**

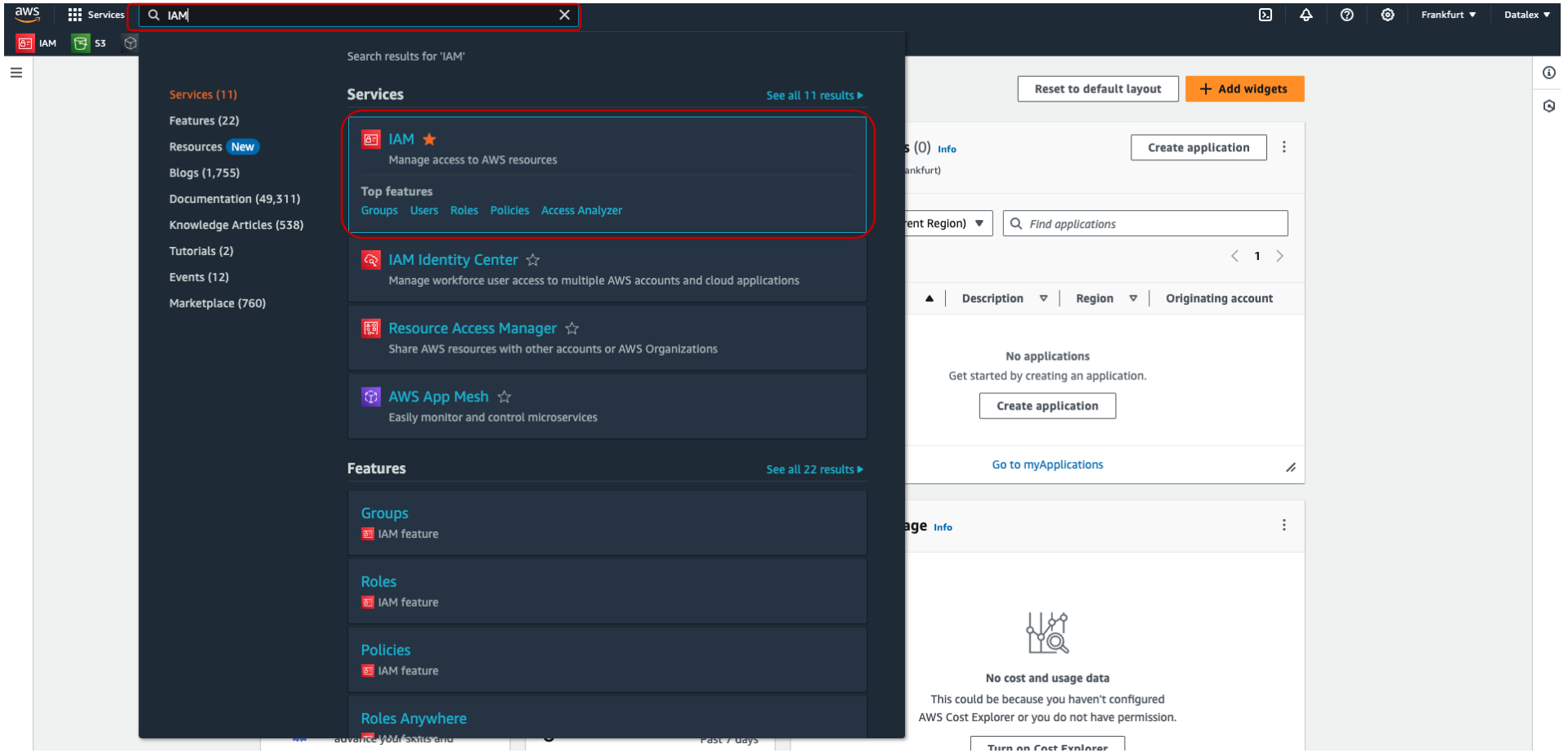
Short-term free trial offers are available through many different software solutions

https://aws.amazon.com/free



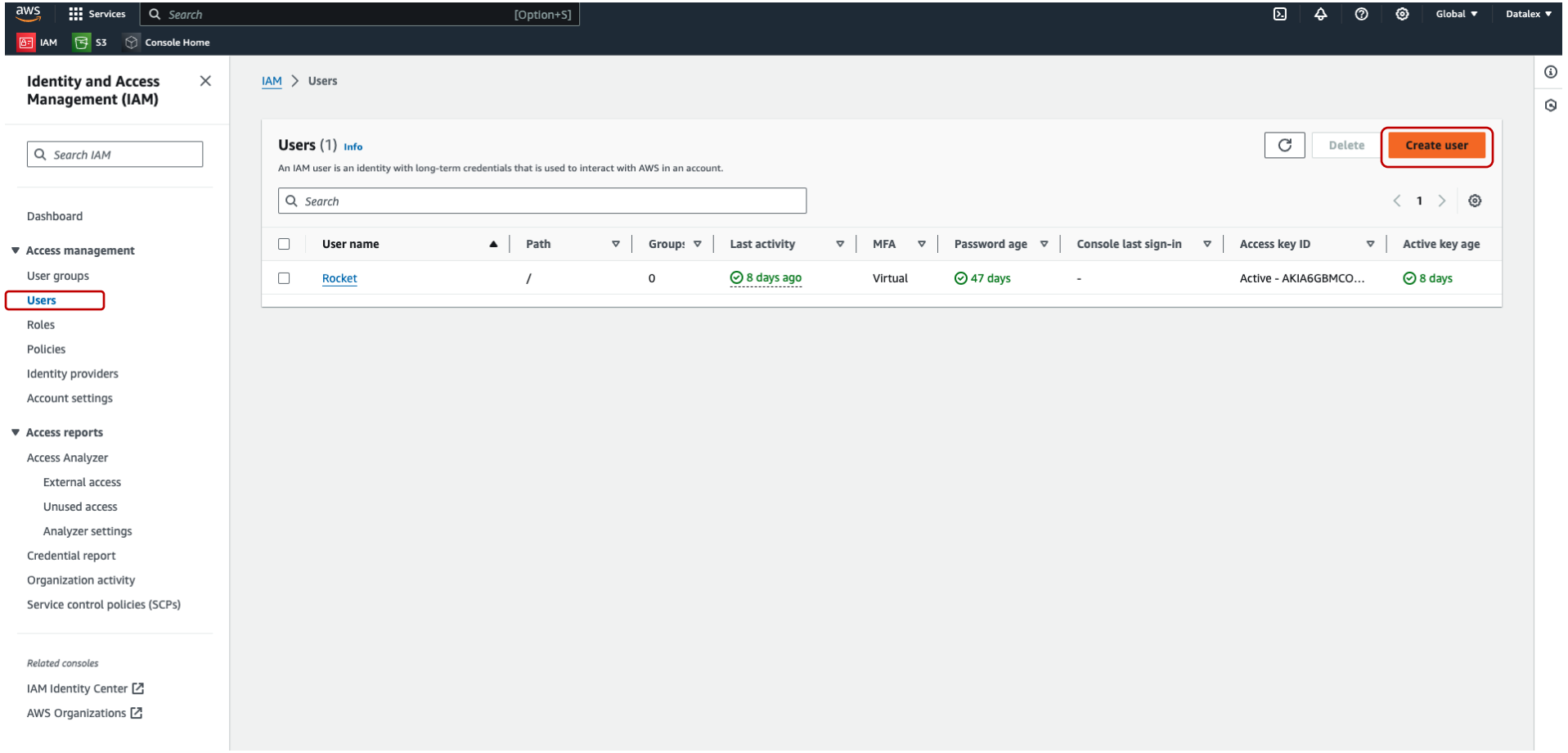
# LAB – AWS Management Console

## Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy



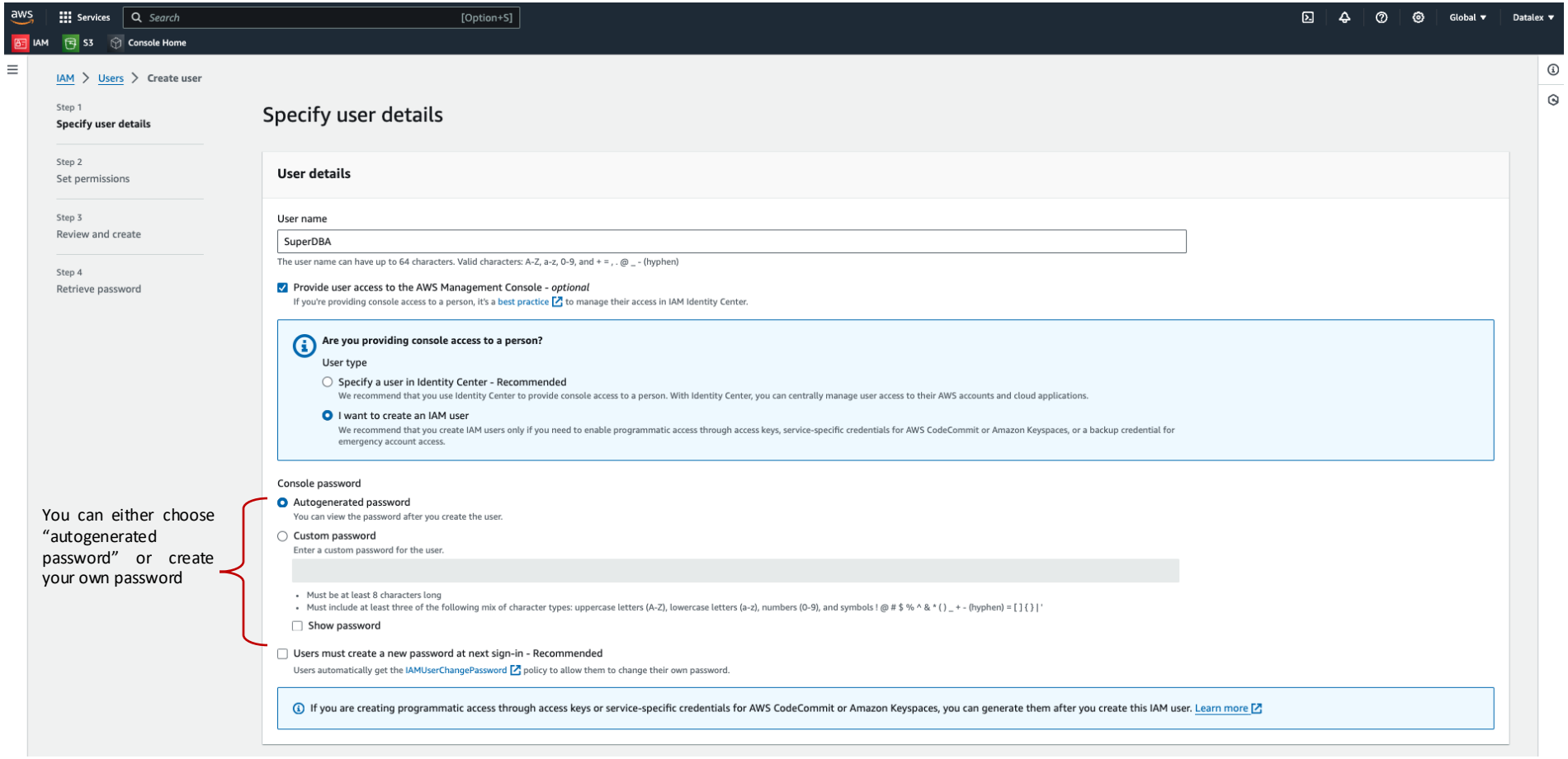
# LAB – AWS Management Console

## Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy



# LAB – AWS Management Console

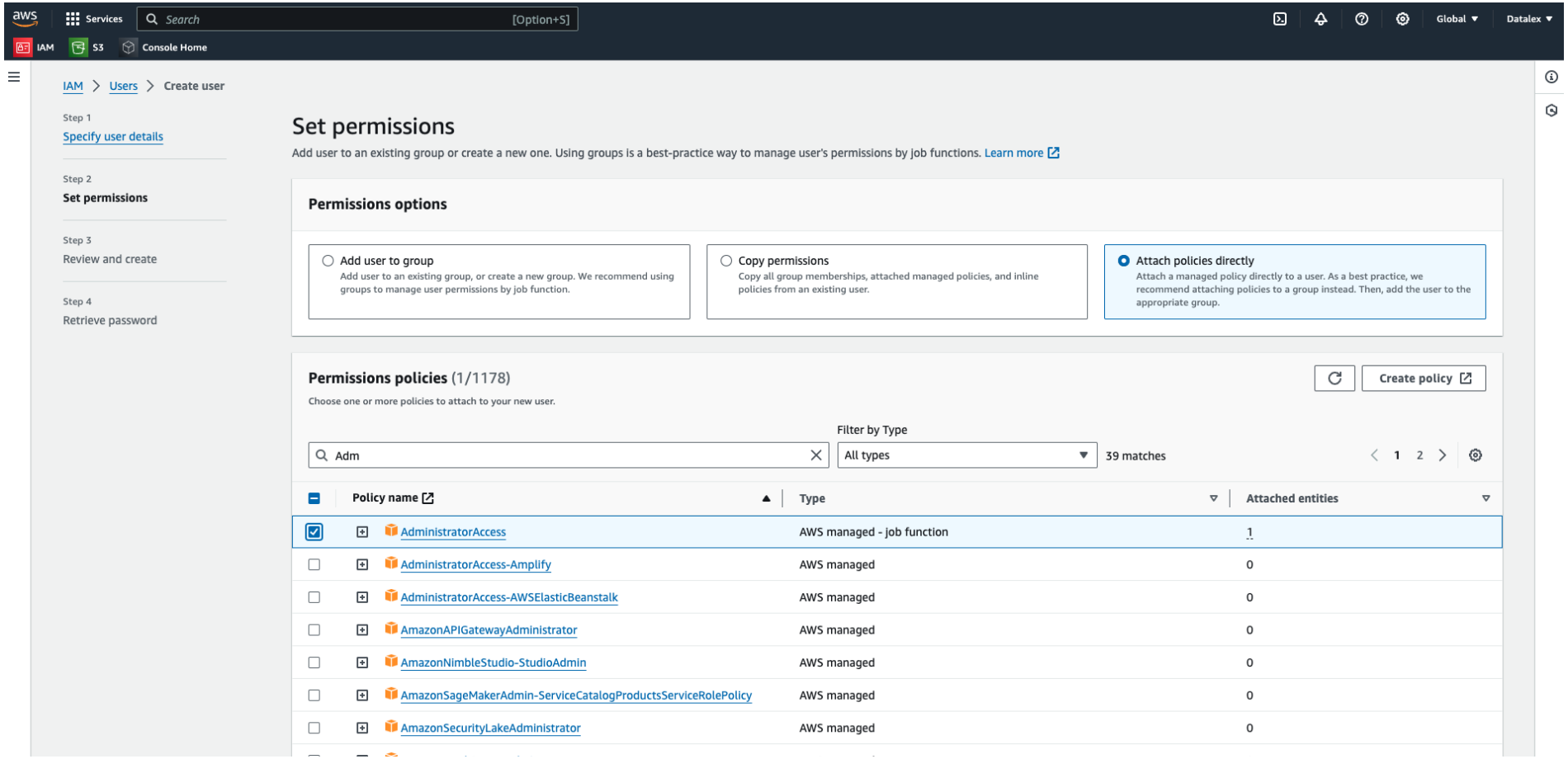
## Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy




You can either choose “autogenerated password” or create your own password

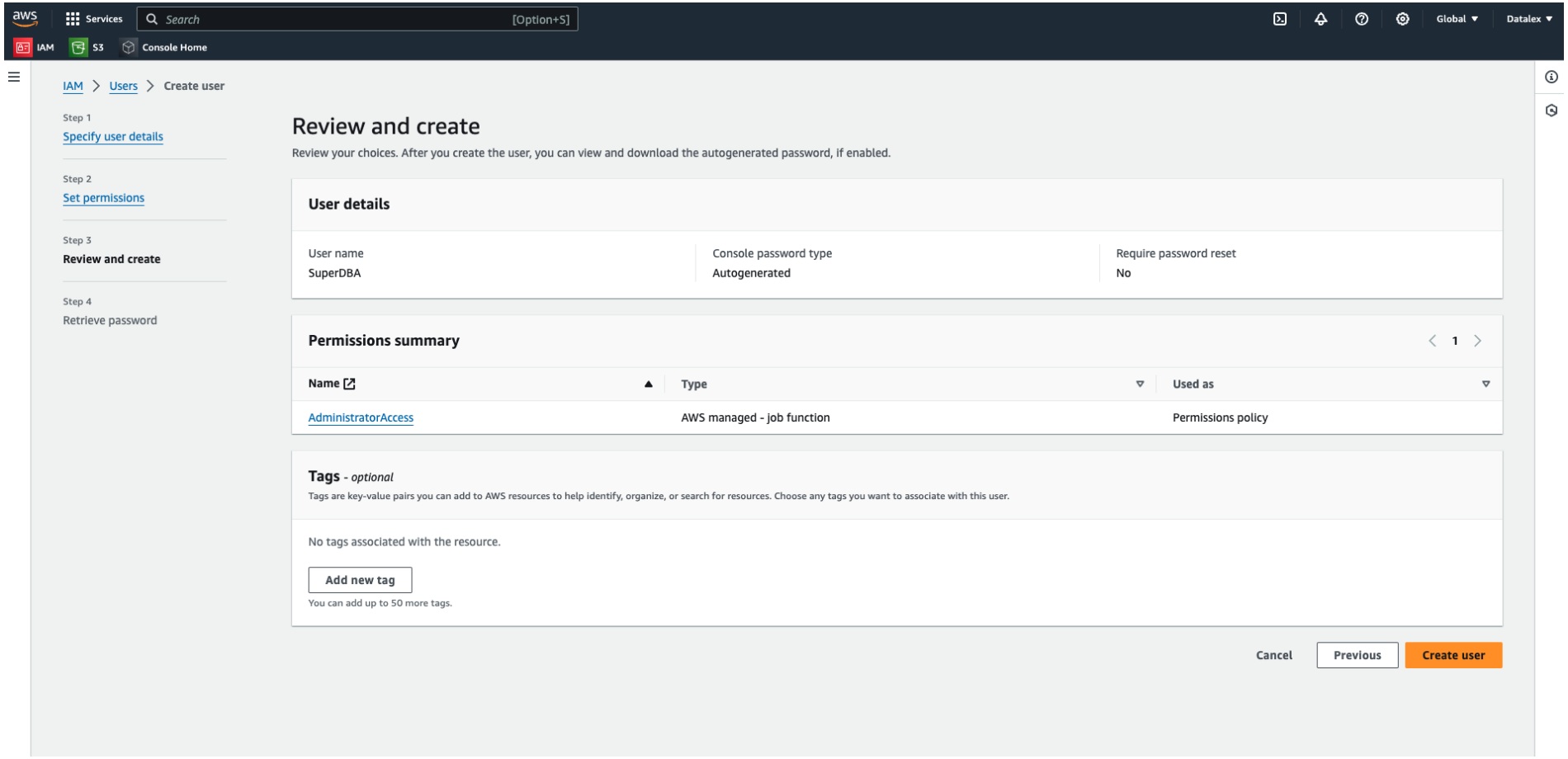
# LAB – AWS Management Console

## Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy



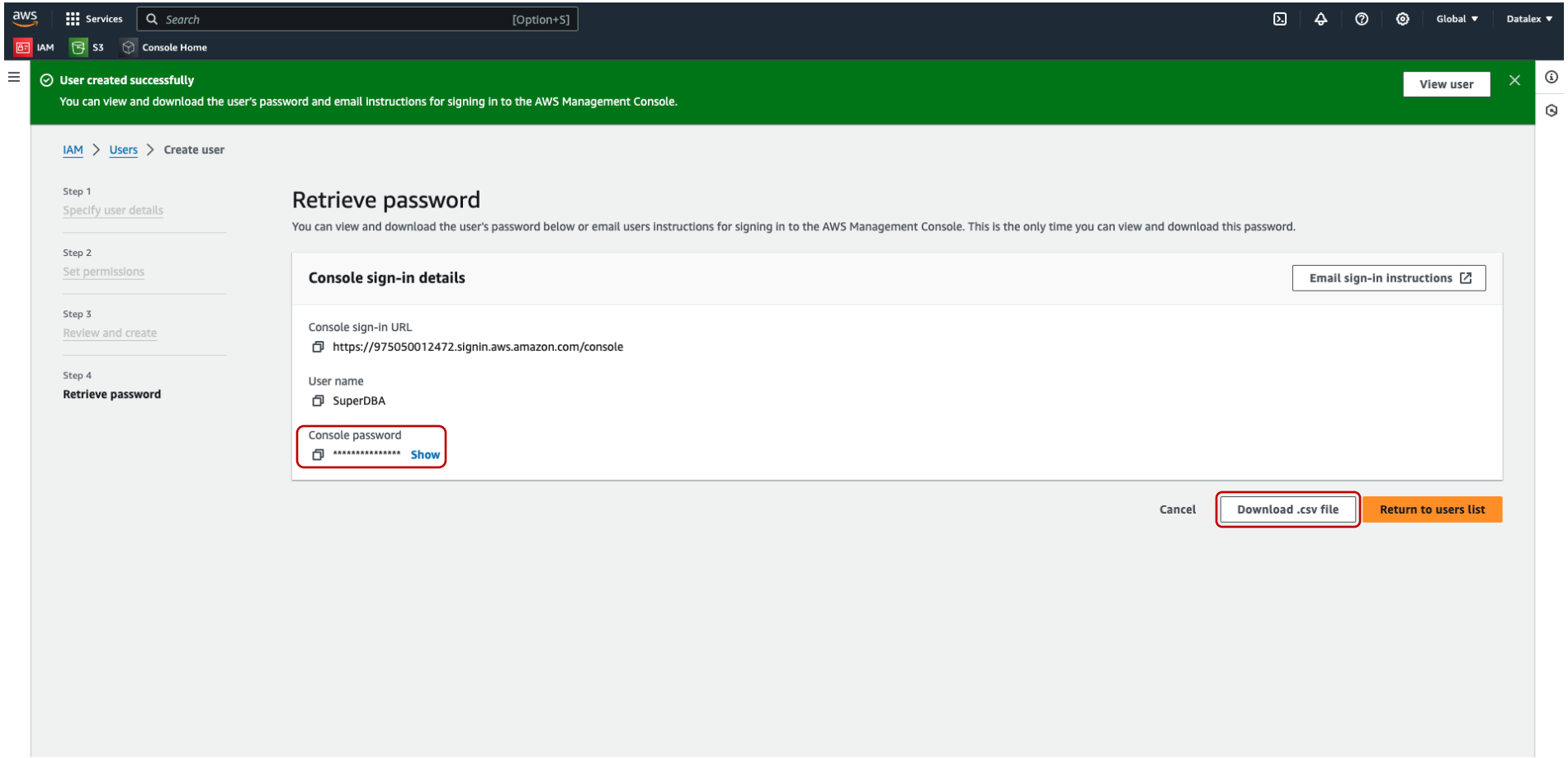
# LAB – AWS Management Console

## Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy



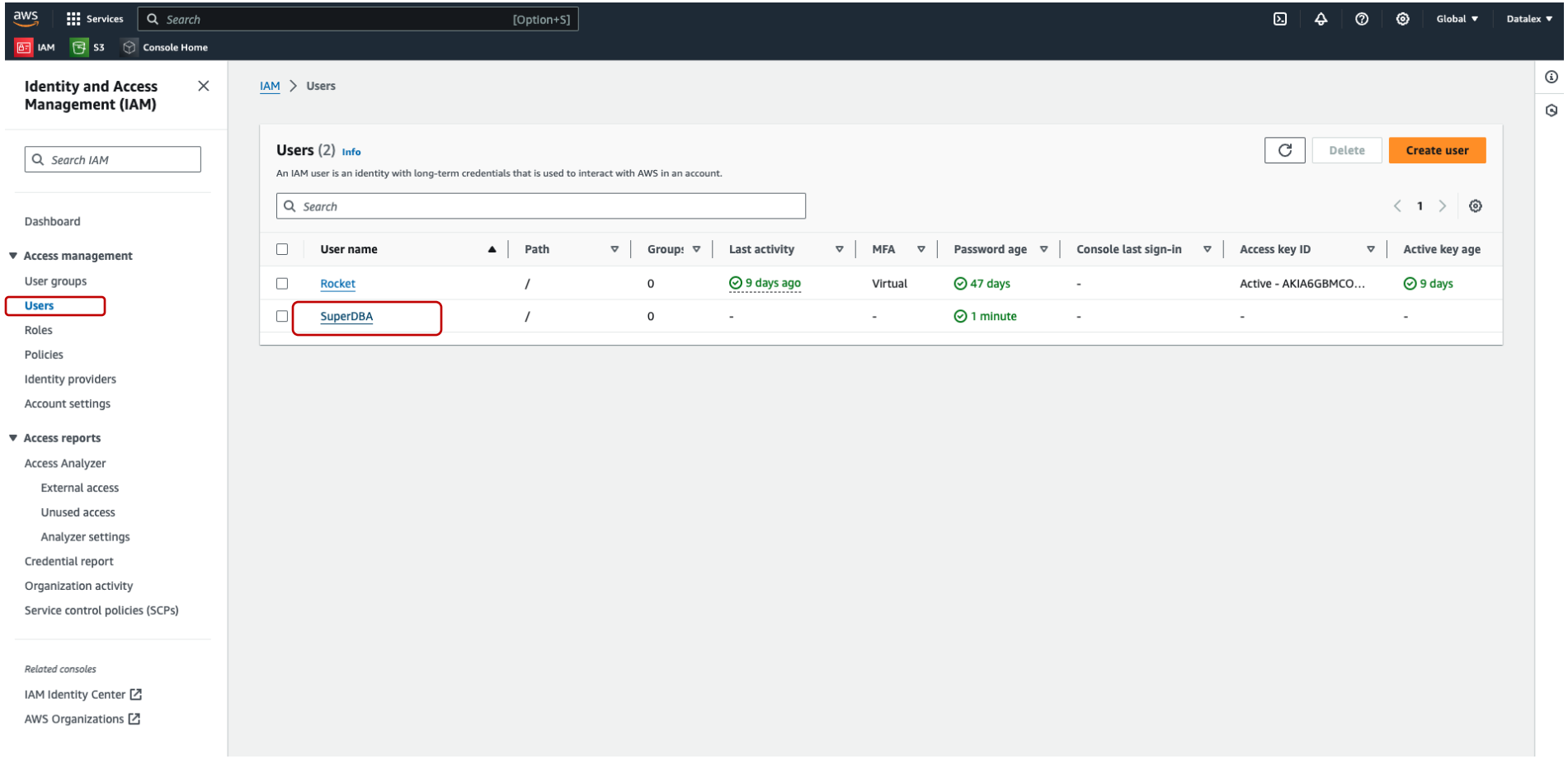
# LAB – AWS Management Console

## Create an AWS IAM User (e.g. admin) & Add « AdministratorAccess » policy



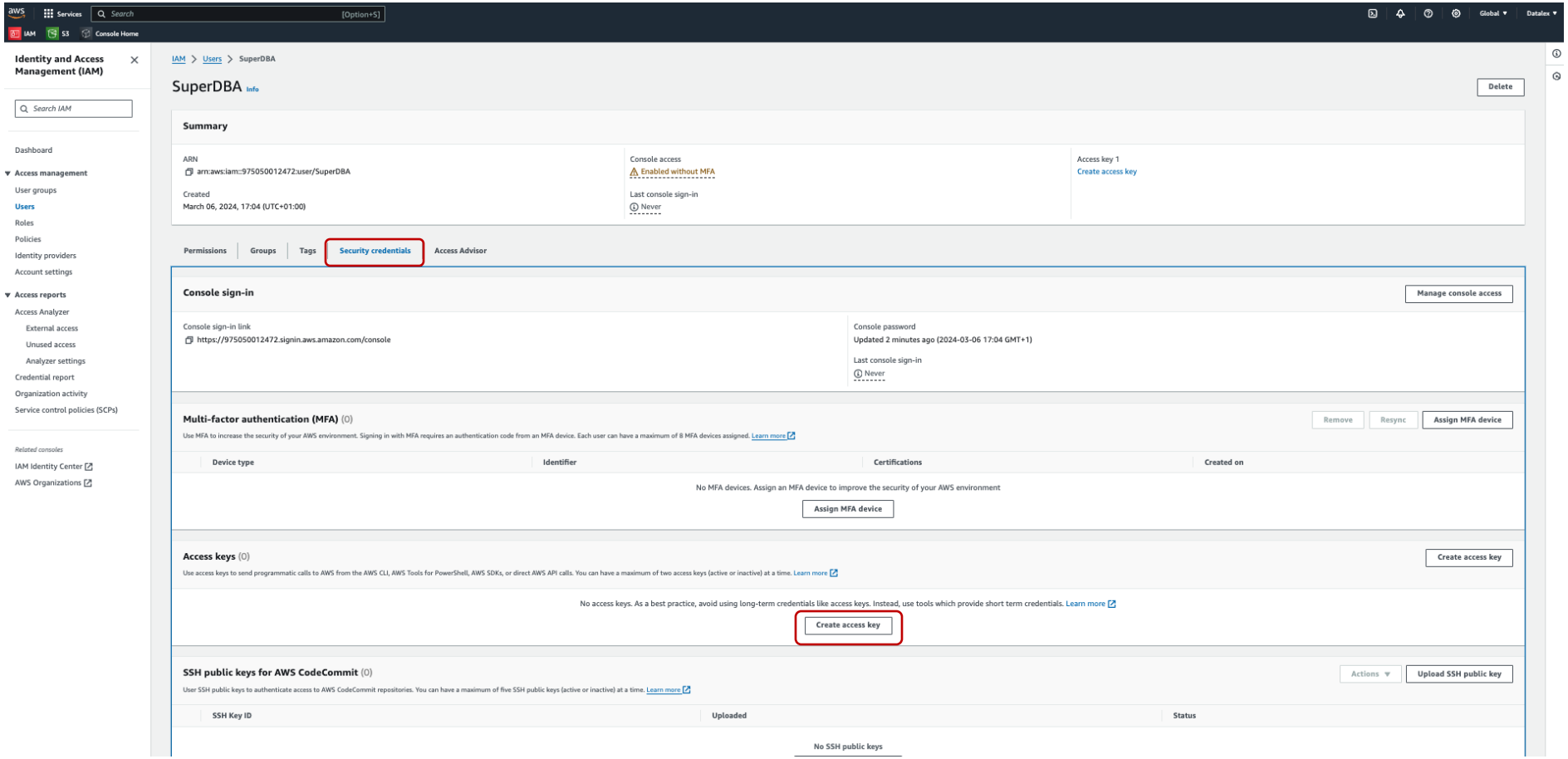
# LAB – AWS Management Console

## Create access key & download the credentials csv file



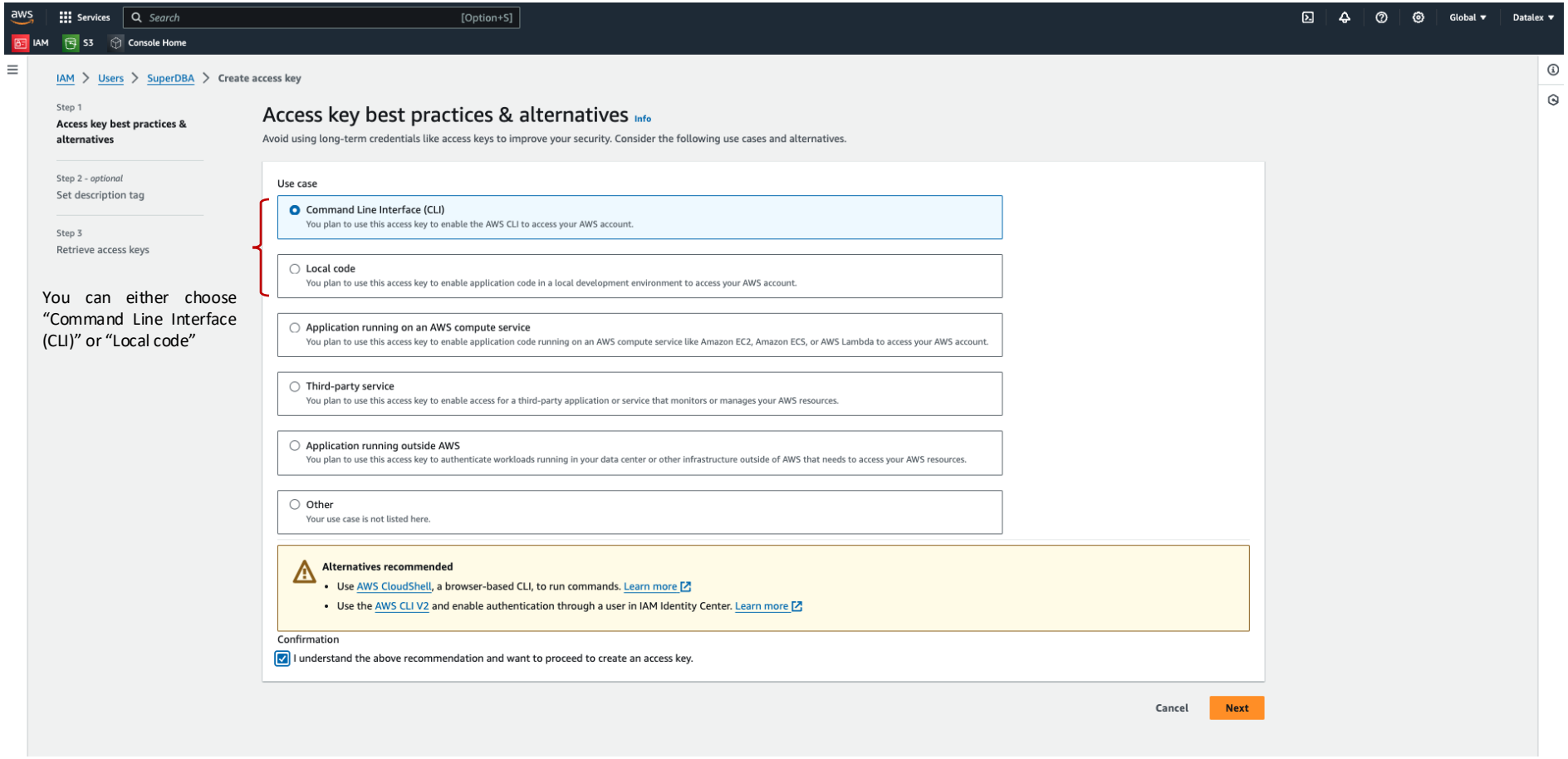
# LAB – AWS Management Console

## Create access key & download the credentials csv file



# LAB – AWS Management Console

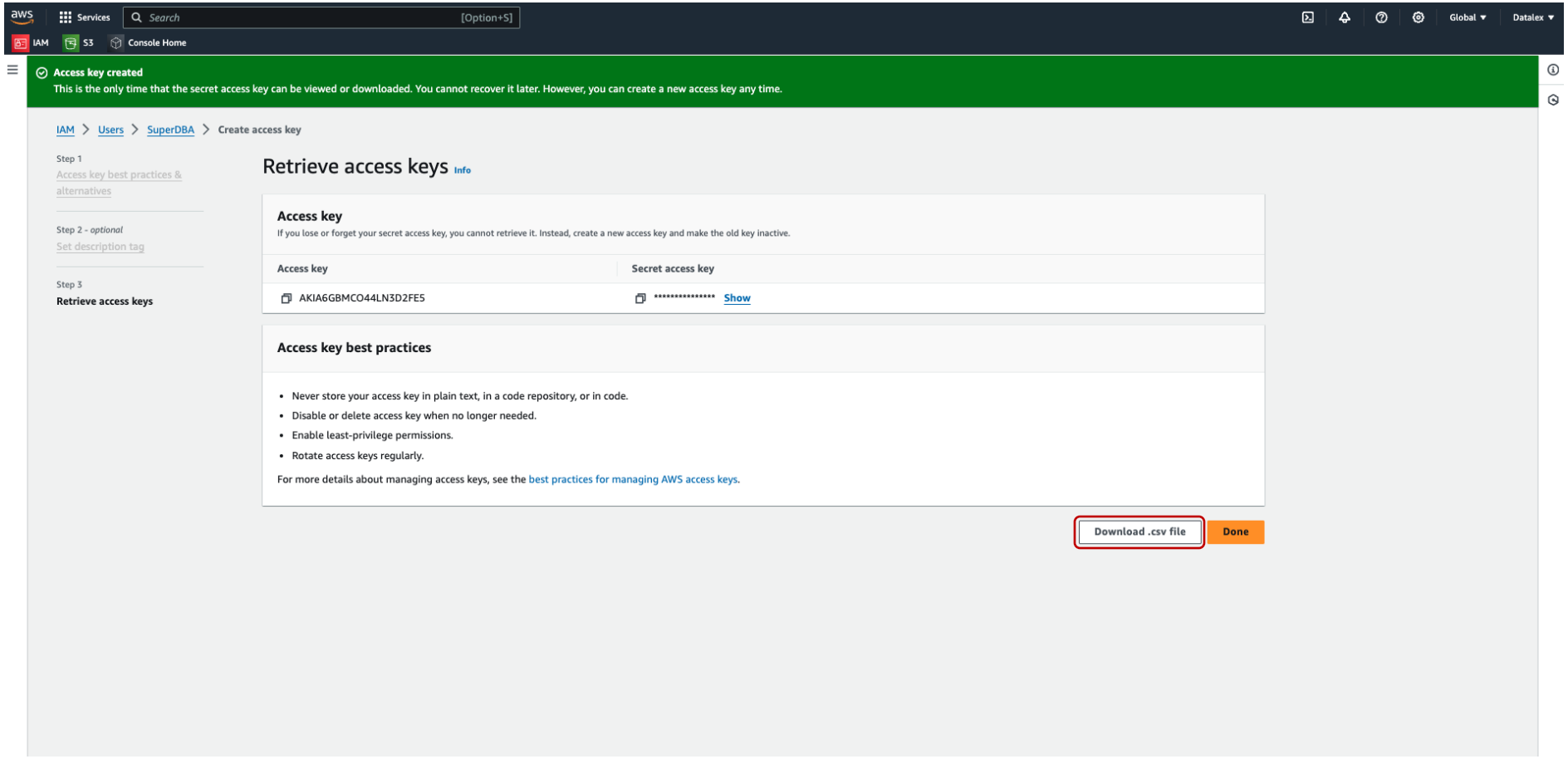
## Create access key & download the credentials csv file




You can either choose “Command Line Interface (CLI)” or “Local code”

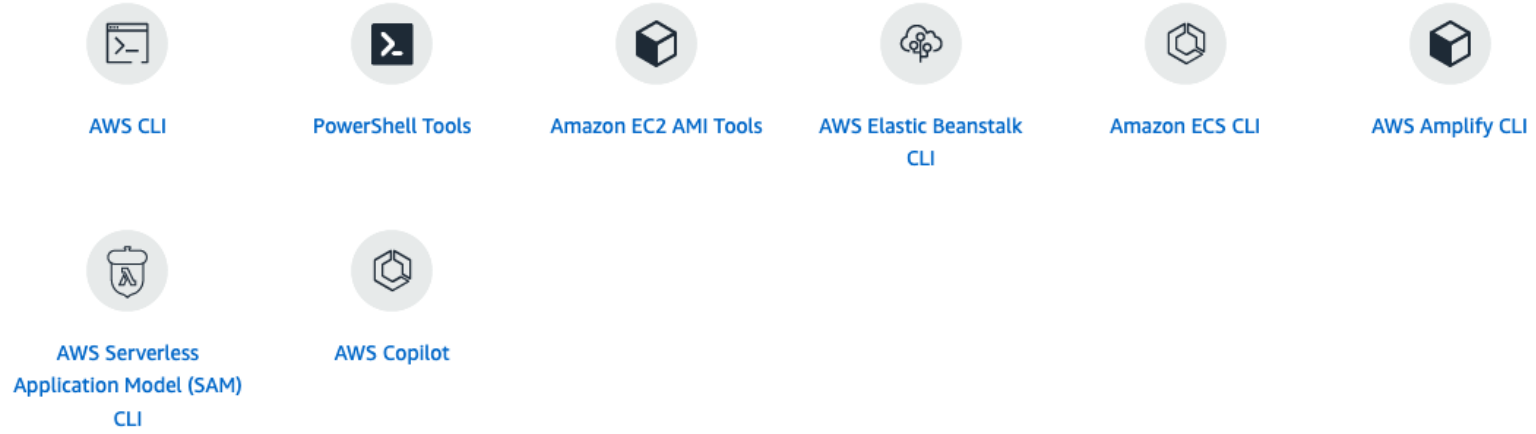
# LAB – AWS Management Console

## Create access key & download the credentials csv file



# AWS CLI

The AWS CLI is a unified tool that you can use to manage AWS services. You can download and configure one tool that you can use to control multiple AWS services from the command line, and automate them with scripts. The AWS CLI is open source, and installers are available for Windows, Linux, and macOS.



# AWS CLI

## installation

### Install and update requirements

- We support the AWS CLI on Microsoft-supported versions of 64-bit Windows.
- Admin rights to install software

### Install or update the AWS CLI

To update your current installation of AWS CLI on Windows, download a new installer each time you update to overwrite previous versions. AWS CLI is updated regularly. To see when the latest version was released, see the AWS CLI version 2 Changelog on *GitHub*.

1. Download and run the AWS CLI MSI installer for Windows (64-bit):
   https://awscli.amazonaws.com/AWSCLIV2.msi
   Alternatively, you can run the `msiexec` command to run the MSI installer.

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

   For various parameters that can be used with `msiexec`, see msiexec on the *Microsoft Docs* website. For example, you can use the `/qn` flag for a silent installation.

```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi /qn
```

2. To confirm the installation, open the **Start** menu, search for `cmd` to open a command prompt window, and at the command prompt use the `aws --version` command.

```
C:\> aws --version
aws-cli/2.15.19 Python/3.11.6 Windows/10 exe/AMD64 prompt/off
```

   If Windows is unable to find the program, you might need to close and reopen the command prompt window to refresh the path, or follow the troubleshooting in Troubleshoot AWS CLI errors.

https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

# AWS CLI

## installation

GUI installer | **Command line installer - All users** | Command line - Current user

If you have `sudo` permissions, you can install the AWS CLI for all users on the computer. We provide the steps in one easy to copy and paste group. See the descriptions of each line in the following steps.

```
$ curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
$ sudo installer -pkg AWSCLIV2.pkg -target /
```

1. Download the file using the `curl` command. The `-o` option specifies the file name that the downloaded package is written to. In this example, the file is written to `AWSCLIV2.pkg` in the current folder.

   ```
   $ curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
   ```

2. Run the standard macOS `installer` program, specifying the downloaded `.pkg` file as the source. Use the `-pkg` parameter to specify the name of the package to install, and the `-target /` parameter for which drive to install the package to. The files are installed to `/usr/local/aws-cli`, and a symlink is automatically created in `/usr/local/bin`. You must include `sudo` on the command to grant write permissions to those folders.

   ```
   $ sudo installer -pkg ./AWSCLIV2.pkg -target /
   ```

   After installation is complete, debug logs are written to `/var/log/install.log`.

3. To verify that the shell can find and run the `aws` command in your `$PATH`, use the following commands.

   ```
   $ which aws
   /usr/local/bin/aws
   $ aws --version
   aws-cli/2.15.19 Python/3.11.6 Darwin/23.3.0 botocore/2.4.5
   ```

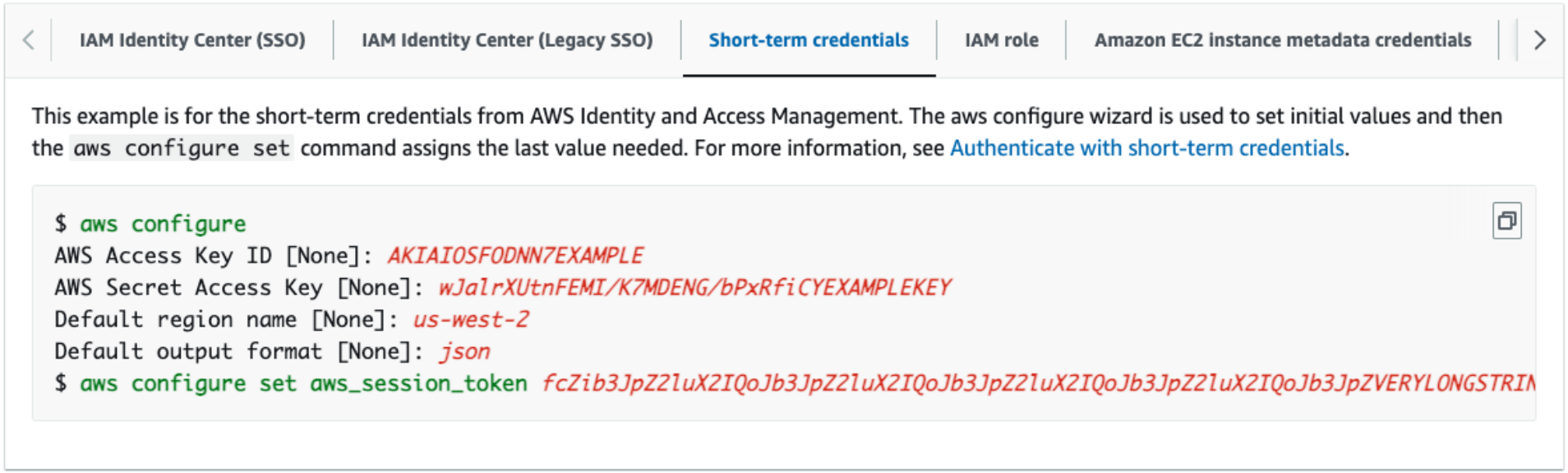
   If the `aws` command cannot be found, you might need to restart your terminal or follow the troubleshooting in Troubleshoot AWS CLI errors.

https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

# AWS CLI

## configuration

```
# List aws cli configuration files
> aws configure
```




https://docs.aws.amazon.com/cli/latest/userguide/getting-started-quickstart.html

# AWS CLI

## Check configuration files

```
# List aws cli configuration files
> ls .aws
```

```
alexandrebergere@alexandres-macbook-pro ~ % ls .aws
cli             config          credentials
alexandrebergere@alexandres-macbook-pro ~ %
```

Configuration and credential file settings: https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html.

# AWS CLI

**config**

```
[default]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = readOnly
region = us-west-2
output = json

[profile dev]
role_arn = arn:aws:iam::444455556666:role/OrganizationAccountAccessRole
source_profile = default

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://my-sso-portal.awsapps.com/start
sso_registration_scopes = sso:account:access
```

# AWS CLI

**credentials**

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

[dev]
aws_access_key_id = AK3RTV675YU9OLPC77
aws_secret_access_key = bcRReOPH879xQttYhRt6Yhg544
```

# AWS CLI

## Check

```
# Lists all users
> aws iam list-users
```


```
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Rocket",
      "UserId": "AIDA6GBMCO44NKTT6LP3R",
      "Arn": "arn:aws:iam::975050012472:user/Rocket",
      "CreateDate": "2024-01-19T08:02:19+00:00"
    }
  ]
}
```

# AWS CLI

## Use profile

```
# # Lists all users using the credentials and settings defined in the user1 profile.
> aws iam list-users --profile user1
```


```
{
    "Users": [
        {
            "Path": "/",
            "UserName": "Rocket",
            "UserId": "AIDA6GBMCO44NKTT6LP3R",
            "Arn": "arn:aws:iam::975050012472:user/Rocket",
            "CreateDate": "2024-01-19T08:02:19+00:00"
        }
    ]
}
```

# LAB – AWS CLI




1. Install and Configure the AWS CLI (Enter the Access Key & Secret Access Key from the credentials.cs)
2. Try “aws iam list-users”
3. Create a new user named « K2000 »

# LAB – AWS CLI

```
# 3. Create a new user named « K2000 »
> aws iam create-user \
   --user-name K2000
```


```
{
        "User": {
                "UserName": "K2000",
                "Path": "/",
                "CreateDate": "2024-02-26T03:13:02.581Z",
                "UserId": "AIDAJY2PE5XUZ4EXAMPLE",
                "Arn": "arn:aws:iam::123456789012:user/K2000"
        }
}
```


Documentation : https://docs.aws.amazon.com/cli/latest/reference/iam/create-user.html

# AWS SDK – Boto3

API calls to AWS can also be performed by running code with programming languages. You can do this by using AWS SDKs. SDKs are open source and maintained by AWS for the most popular programming languages, such as C++, Go, Java, JavaScript, .NET, Node.js, PHP, Python, Ruby, Rust, and Swift.

```
pip install boto3
```

https://aws.amazon.com/fr/developer/tools/

# AWS SDK – Boto3

## Client

To interact with a specific AWS service, you need to create a Boto3 client for that service.

For example, if you want to work with Amazon EC2:

```
ec2_client = boto3.client('ec2')
```

This creates a client for the Amazon EC2 service, and ec2_client can be used to make requests to EC2.

# AWS SDK – Boto3

## Client

Amazon EC2 Client Example:

list_instances.py

```python
import boto3

# Create an EC2 client
ec2_client = boto3.client('ec2')

# Describe all EC2 instances
response = ec2_client.describe_instances()
instances = response['Reservations']

print('EC2 Instances:')
for reservation in instances:
    for instance in reservation['Instances']:
        print('Instance ID:', instance['InstanceId'])
        print('Instance State:', instance['State']['Name'])
```

# AWS SDK – Boto3

## Client

Amazon S3 Client Example:

list_buckets.py

```python
import boto3

# Create an S3 client
s3_client = boto3.client('s3')

# List all S3 buckets
response = s3_client.list_buckets()
buckets = response['Buckets']

print('S3 Buckets:')
for bucket in buckets:
    print(bucket['Name'])
```

# AWS SDK – Boto3

## Client

AWS Identity and Access Management (IAM) Client Example:

list_users.py

```python
import boto3

# Create an IAM client
iam_client = boto3.client('iam')

# List all IAM users
response = iam_client.list_users()
users = response['Users']

print('IAM Users:')
for user in users:
    print('Username:', user['UserName'])
```

# AWS SDK – Boto3

## Authentication

To use authentication with Boto3, you need to provide your AWS credentials. Boto3 supports various methods for authentication, including using access keys, session tokens, or profiles configured in your AWS CLI configuration.

Configure AWS Credentials - There are a few ways to configure your AWS credentials for Boto3:

- Environment Variables
- Configuration Files
- Using Boto3 Directly

# AWS SDK – Boto3

## Authentication

Environment Variables:

```
export AWS_ACCESS_KEY_ID=your_access_key_id
export AWS_SECRET_ACCESS_KEY=your_secret_access_key
```

Configuration Files:

```
aws configure
```

Using Boto3 Directly:

```
import boto3
# Replace 'your_access_key_id' and 'your_secret_access_key' with your actual credentials
client = boto3.client('ec2', aws_access_key_id='your_access_key_id', aws_secret_access_key='your_secret_access_key')
```

# AWS SDK – Boto3

## Authentication

Configuration Files - You can specify the profile you want to use.:

```
import boto3

# Use the desired AWS CLI profile
session = boto3.Session(profile_name='your_profile_name')

# Now, you can create clients and resources using this session
s3_client = session.client('s3')
ec2_resource = session.resource('ec2')

# Perform operations using these clients and resources
buckets = s3_client.list_buckets()
instances = ec2_resource.instances.all()
```

# LAB – AWS BOTO3




1. Install and Configure boto3.
2. Add credentials to boto client.
3. Create a new user named « sarah.connor ».
4. Delete user named « sarah.connor ».

Documentation :

- https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/iam/client/create_user.html
- https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/iam/client/delete_user.html

# AWS SDK – Boto3

## Authentication

```python
from typing import Optional
import boto3
from botocore.credentials import InstanceMetadataProvider, InstanceMetadataFetcher

valid_authentication_type = ["instance_profile", "iam", "assume_role"]

class AWSCredentials:
    authentication_type: str
    aws_access_key_id: str
    aws_secret_access_key: str
    aws_token: Optional[str] = None

    def __init__(self, authentication_type: str, aws_access_key_id: str = None, aws_secret_access_key: str = None, RoleArn=None, RoleSessionName=None):
        if authentication_type not in valid_authentication_type:
            raise ValueError(f"`authentication_type` must be one of ${', '.join(valid_authentication_type)}")

        self.authentication_type = authentication_type

        if (self.authentication_type == "instance_profile"):
            provider = InstanceMetadataProvider(iam_role_fetcher=InstanceMetadataFetcher(timeout=1000, num_attempts=2))
            creds = provider.load().get_frozen_credentials()
            self.aws_access_key_id = creds.access_key
            self.aws_secret_access_key = creds.secret_key
            self.aws_token = creds.token
        else:
            if (self.authentication_type == "iam"):
                self.aws_access_key_id = aws_access_key_id
                self.aws_secret_access_key = aws_secret_access_key
            else:
                provider = boto3.client('sts').assume_role(RoleArn=RoleArn, RoleSessionName=RoleSessionName)
                creds = provider['Credentials']
                self.aws_access_key_id = creds['AccessKeyId']
                self.aws_secret_access_key = creds['SecretAccessKey']
                self.aws_token = creds['SessionToken']


def get_authentified_client(client: str, creds: AWSCredentials):
    return boto3.client(client, aws_access_key_id=creds.aws_access_key_id, aws_secret_access_key=creds.aws_secret_access_key)


def get_authentified_session(creds: AWSCredentials):
    return boto3.Session(aws_access_key_id=creds.aws_access_key_id, aws_secret_access_key=creds.aws_secret_access_key, aws_session_token=creds.aws_token)
```

# LAB – AWS BOTO3

```
> import boto3

# 2. Add credentials to boto client.
> client = boto3.client('iam', aws_access_key_id=creds.aws_access_key_id,
aws_secret_access_key=creds.aws_secret_access_key)

Or

> client = boto3.client(aws_access_key_id=creds.aws_access_key_id,
aws_secret_access_key=creds.aws_secret_access_key).client('iam')

Or

> client = boto3.client('iam')
```

# LAB – AWS BOTO3

```
# 3. Create a new user named « sarah.connor ».
> client.create_user(UserName='sarah.connor')


# 4. Delete user named « sarah.connor »
> client.delete_user(UserName='sarah.connor')
```

# AWS

## S3



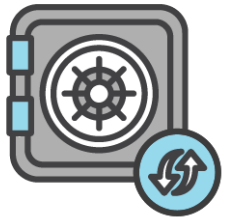
# Amazon S3



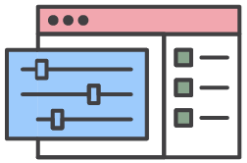
Amazon S3



Object-level storage



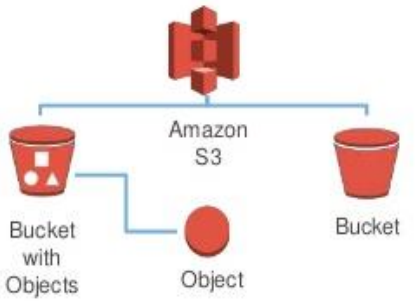
Designed for 99.999999999% durability



Event triggers

# Amazon S3

## Amazon S3 Concepts




- Amazon S3 stores data as objects within **buckets**
- An object is composed of a file and optionally any **metadata** that describes that file
- You can have **up to 100 buckets** in each account
- You can **control access** to the bucket and its objects

An object key is the unique identifier for an object in a bucket.



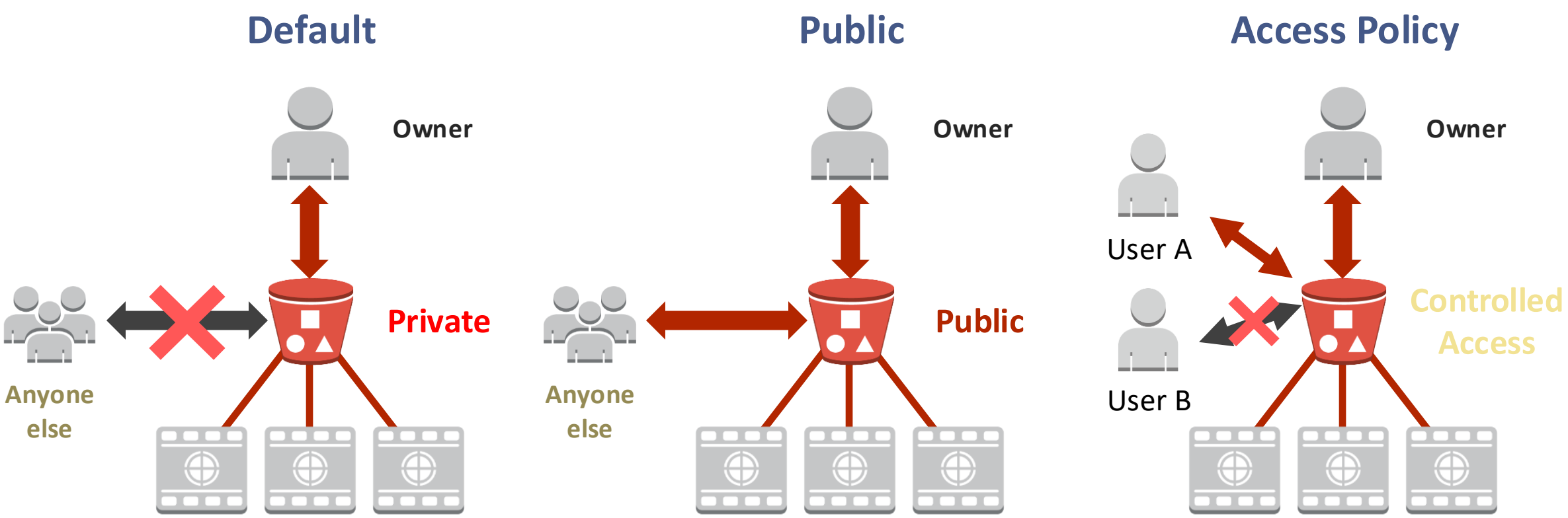
# Amazon S3

## Use cases

- **Backup and storage**: Amazon S3 is a natural place to back up files because it is highly redundant. As mentioned in the last lesson, AWS stores your EBS snapshots in Amazon S3 to take advantage of its high availability.
- **Media hosting:** Because you can store unlimited objects, and each individual object can be up to 5 TB, Amazon S3 is an ideal location to host video, photo, and music uploads.
- **Software delivery**: You can use Amazon S3 to host your software applications that customers can download.
- **Data lakes**: Amazon S3 is an optimal foundation for a data lake because of its virtually unlimited scalability. You can increase storage from gigabytes to petabytes of content, paying only for what you use.
- **Static web**: You can configure your S3 bucket to host a static website of HTML, CSS, and client-side scripts.
- **Static content**: Because of the limitless scaling, the support for large files, and the fact that you can access any object over the web at any time, Amazon S3 is the perfect place to store static content.

# Amazon S3

## Access Control - General



# Amazon S3

## Access Control - Bucket Policies



**Bucket Policies**



```
{
  "Version": "2012-10-17",
  "Id": "S3PolicyId1",

  "Statement": [

    {
      "Sid": "IPAllow",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": "arn:aws:s3:::examplebucket/*",
      "Condition": {
         "NotIpAddress": {"aws:SourceIp":
"54.240.143.0/24"}

      }
    }
  ]
}
```

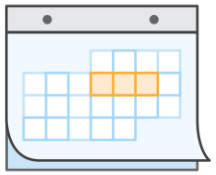
AWS Policy Generator

# Amazon S3

## Glacier



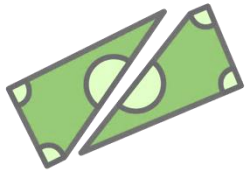
Amazon S3 Glacier



Long-term data storage



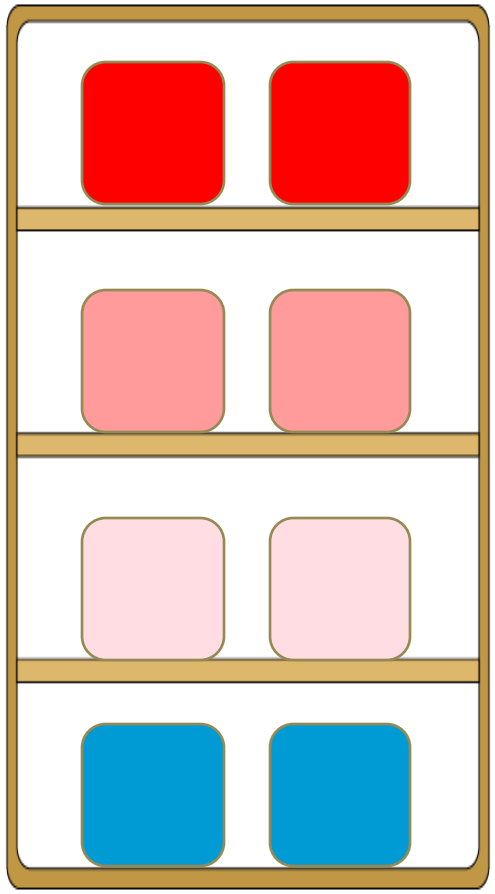
Archival or backup



Very low-cost storage

# Amazon S3

## Tiering



**S3 Standard:**
Frequently accessed data

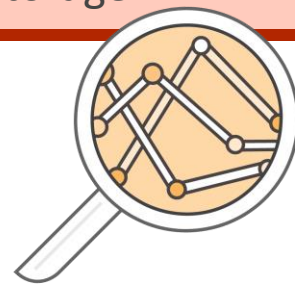
**S3 Standard IA:**
Long-lived, infrequently accessed data

**S3 One Zone IA:**
Long-lived, infrequent, but rapid access

**Amazon Glacier / Deep Archive:**
Archiving rarely accessed data

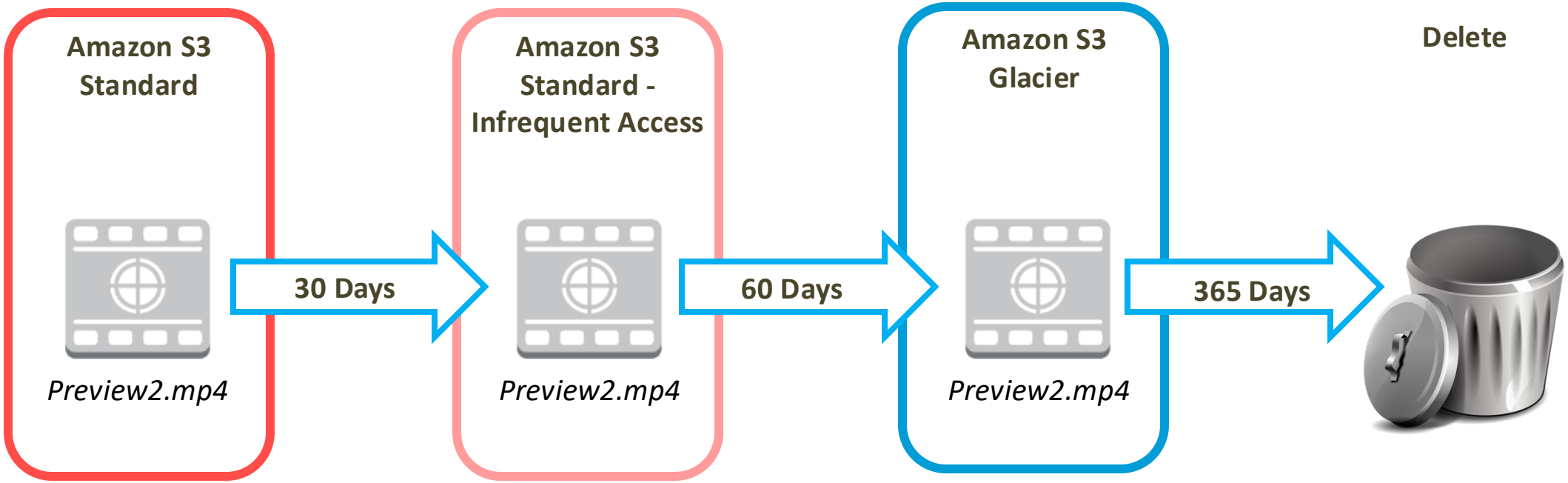
**Amazon S3 Intelligent Tiering**

Automatically moves your objects between two access tiers of storage



# Amazon S3

## Lifecycle Policies



# S3 Documentation




https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

# Amazon S3

Regions outside of us-east-1 require the appropriate LocationConstraint to be specified in order to create the bucket in the desired region.

```
# Create bucket
> bucket_name='esme-s3-xb37-bucket'
> s3_client.create_bucket(Bucket=bucket_name, CreateBucketConfiguration={'LocationConstraint': 'eu-central-1'})
```

Source: https://docs.aws.amazon.com/cli/latest/reference/s3api/create-bucket.html.

# LAB – AWS S3




With AWS CLI:

1. Creating a bucket named esme-s3-<(4)hexadecimal>-bucket.
2. Upload the 3 lift csv files in a folder called « lyft ».
3. Download back riders.csv from s3 to your local computer.
4. List files inside bucket.
5. Delete all files.
6. Delete esme-s3-bucket.

With boto3:

1. Creating a bucket named esme-s3-<(4)hexadecimal>-bucket.
2. Upload the 3 lift csv files in a folder called « lyft ».
3. Download back riders.csv from s3 to your local computer.
4. List files inside bucket.
5. Delete all files.
6. Delete esme-s3-bucket.

Documentation:
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3

# LAB – AWS S3 - CLI

```
# 1. Creating a bucket named esme-s3-<(4)hexadecimal>-bucket.
> aws s3api create-bucket --bucket 'esme-s3-xb37-bucket' --region eu-central-1 --create-bucket-configuration
LocationConstraint=eu-central-1
```

```
{
   "Location": "http://esme-s3-xb37-bucket.s3.amazonaws.com/"
}
```

# LAB – AWS S3 - CLI

```
# 2. Upload the 3 lift csv files in a folder called « lyft ».
> aws s3 cp 'data/lyft' s3://esme-s3-xb37-bucket/lyft --recursive

# 3. Download back riders.csv from s3 to your local computer.
> aws s3 cp s3://esme-s3-xb37-bucket/lyft/ . --recursive

# 4. List files inside bucket.
> aws s3 ls s3://esme-s3-xb37-bucket/ --recursive
> aws s3 ls s3://esme-s3-xb37-bucket/lyft/

# 5. Delete all files.
# removing single file
> aws s3 rm s3://esme-s3-xb37-bucket/lyft/cars.csv
# removing multiple files
> aws s3 rm s3://esme-s3-xb37-bucket --exclude "*" --include "lyft/*" --recursive

# 6. Delete esme-s3-bucket.
# empty bucket
> aws s3 rm s3://esme-s3-xb37-bucket --recursive
# destroy bucket (Once all the objects have been deleted and the S3 bucket is empty)
> aws s3 rb s3://esme-s3-xb37-bucket
```

# LAB – AWS S3 – BOTO3

```
> import boto3

# 1. Creating a bucket named esme-s3-<(4)hexadecimal>-bucket.
> s3_client = boto3.client('s3', aws_access_key_id=creds.aws_access_key_id,
aws_secret_access_key=creds.aws_secret_access_key)

Or

> session = boto3.Session(profile_name='datalex’)
> s3_client = session.client('s3')

# Create bucket
> bucket_name='esme-s3-xb37-bucket'
> s3_client.create_bucket(Bucket=bucket_name, CreateBucketConfiguration={'LocationConstraint': 'eu-central-1'})
```

Documentation:
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/create_bucket.html

# LAB – AWS S3 – BOTO3

```
# 2. Upload the 3 lift csv files in a folder called « lyft ».
> with open('data/lyft/riders.csv', 'rb') as f:
    s3_client.upload_fileobj(f, bucket_name, 'lyft/riders.csv')


> s3_client.upload_file('data/lyft/cars.csv', bucket_name, 'lyft/cars.csv')
> s3_client.upload_file('data/lyft/trips.csv', bucket_name, 'lyft/trips.csv')


# 3. Download back riders.csv from s3 to your local computer.
> s3_client.download_file(bucket_name, 'lyft/riders.csv', 'riders.csv')
> print(open('riders.csv').read())
```

# LAB – AWS S3 – BOTO3

```
# 4. List files inside bucket.
> print(s3_client.list_buckets())

# 5. Delete all files.
> s3_client.delete_object(Bucket=bucket_name, Key='lyft/cars.csv')
> s3_client.delete_object(Bucket=bucket_name, Key='lyft/riders.csv')
> s3_client.delete_object(Bucket=bucket_name, Key='lyft/trips.csv')

# 6. Delete esme-s3-bucket.
> s3_client.delete_bucket(Bucket=bucket_name)
```

# LAB – AWS S3: Host a Static Website



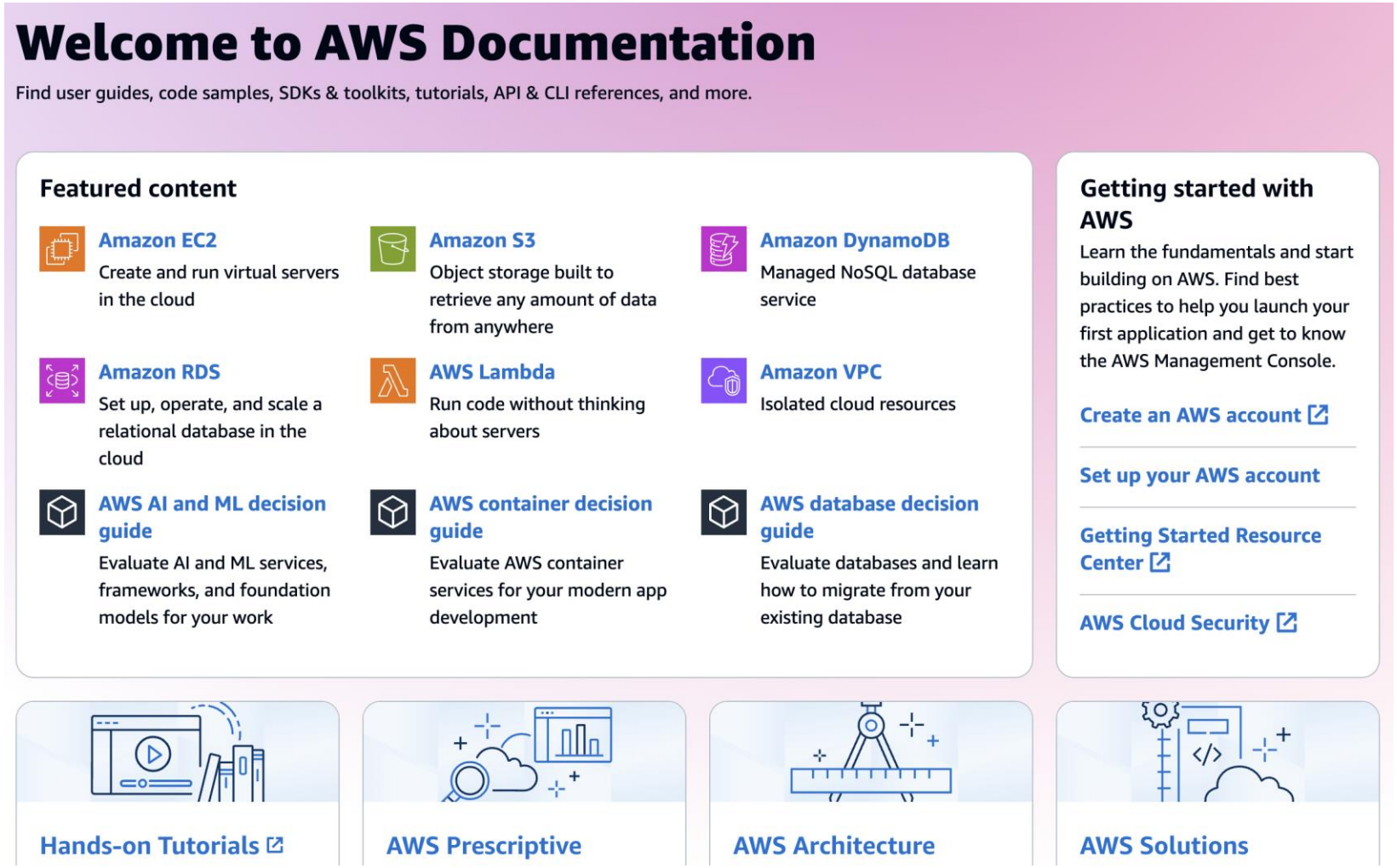

1. Follow this hands-on: Configuring a static website on Amazon S3
2. [Optionnal] Follow this hands-on: Configuring a static website using a custom domain registered with Route 53
3. [Optionnal] Follow this hands-on: Host a Static Website with AWS Amplify
4. [Optionnal] Follow this hands-on: Hosting on-demand streaming video with Amazon S3, Amazon CloudFront, and Amazon Route 53

# AWS

## Documentation

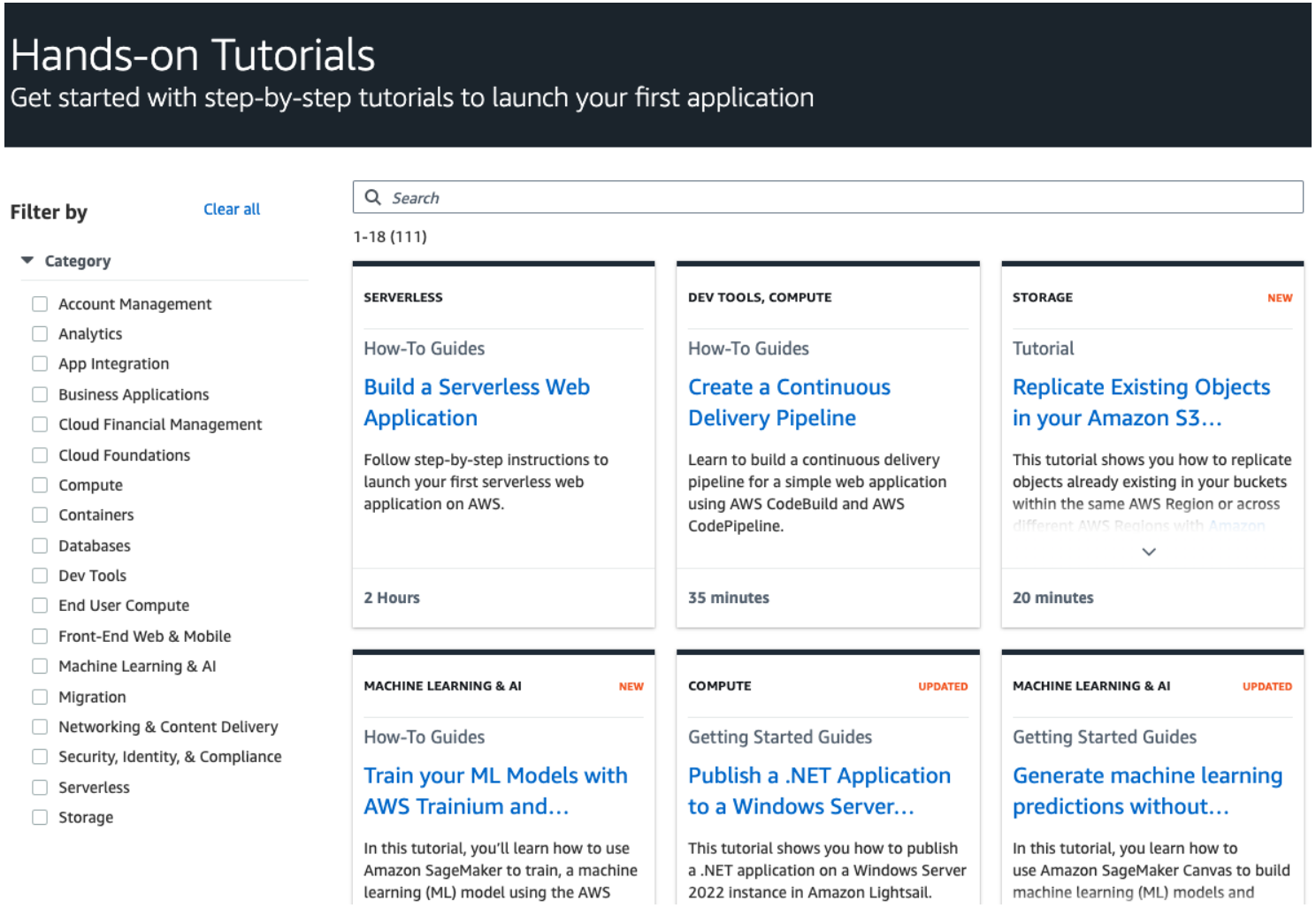


# AWS Documentation



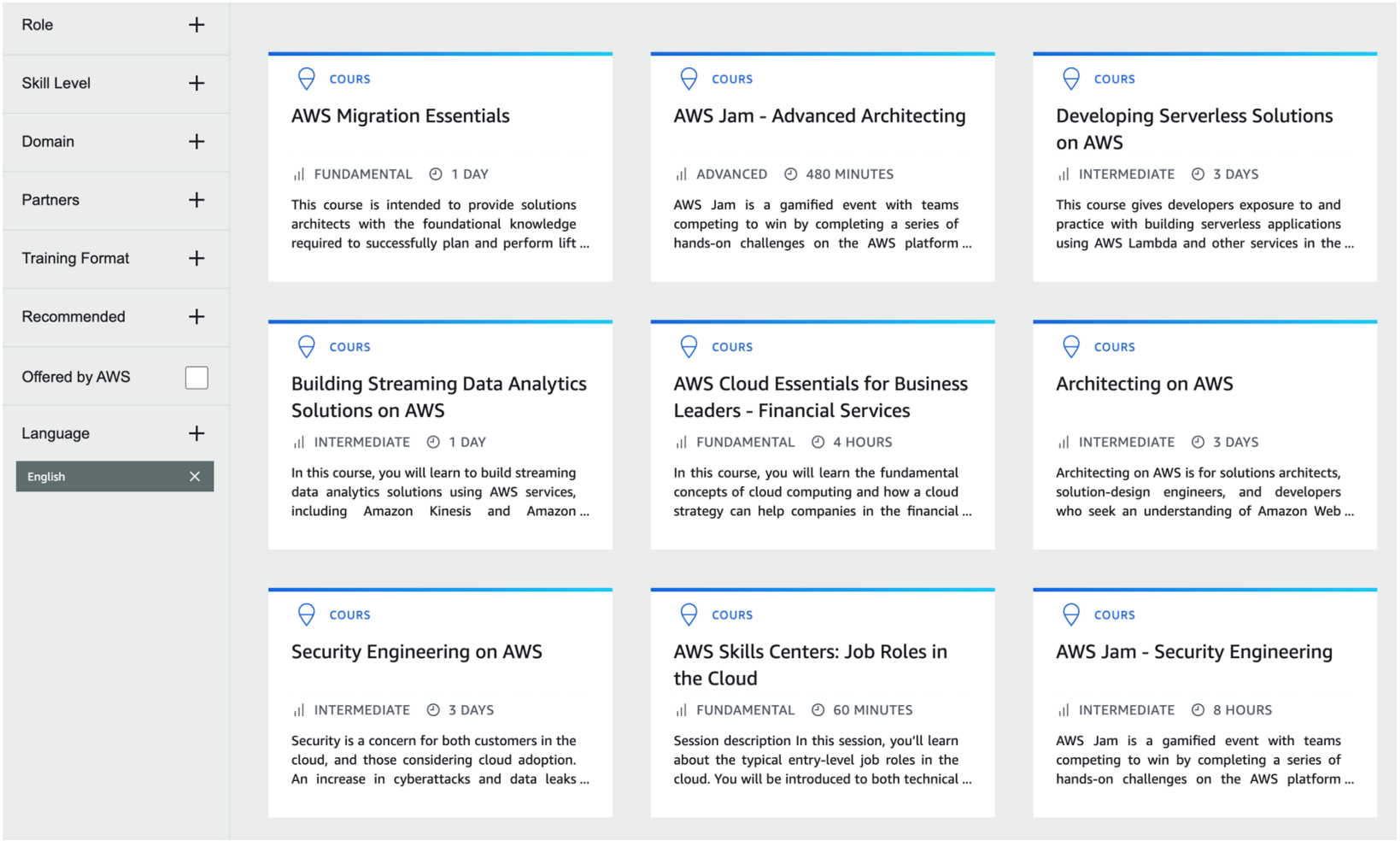

https://docs.aws.amazon.com/

# Getting start on AWS




https://aws.amazon.com/getting-started/hands-on

# AWS - training




https://www.aws.training/LearningLibrary?src=path-arc

# AWS – Architecture blog

**AWS Architecture Blog**

## Category: Analytics

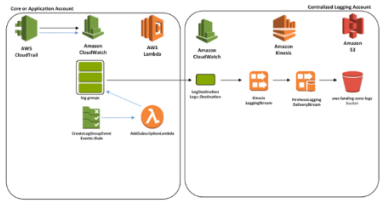
This is My Architecture

### This Is My Architecture: Mobile Cryptocurrency Mining

by Annik Stahl | on 27 MAR 2019 | in Amazon API Gateway, Amazon Athena, Amazon DynamoDB, Architecture, AWS Lambda, AWS WAF, Blockchain, Innovation And Reinvention | Permalink | Comments | Share

In North America, approximately 95% of adults over the age of 25 have a bank account. In the developing world, that number is only about 52%. Cryptocurrencies can provide a platform for millions of unbanked people in the world to achieve financial freedom on a more level financial playing field. Electroneum, a cryptocurrency company located […]

Read More



### Stream Amazon CloudWatch Logs to a Centralized Account for Audit and Analysis

by David Bailey | on 03 DEC 2018 | in Amazon CloudWatch, Amazon Kinesis, Architecture, AWS Identity And Access Management (IAM), AWS Lambda, AWS SDK For Python, How-To | Permalink | Share

A key component of enterprise multi-account environments is logging. Centralized logging provides a single point of access to all salient logs generated across accounts and regions, and is critical for auditing, security and compliance. While some customers use the built-in ability to push Amazon CloudWatch Logs directly into Amazon Elasticsearch Service for analysis, others would […]

Read More



### Store, Protect, Optimize Your Healthcare Data with AWS: Part 2

by Stephen Jepsen | on 04 OCT 2018 | in Amazon Athena, Amazon QuickSight, Amazon Redshift, Analytics, Architecture, Artificial Intelligence, AWS Glue, SageMaker | Permalink | Share

Leveraging Analytics and Machine Learning Tools for Readmissions Prediction This blog post was co-authored by Ujjwal Ratan, a senior AI/ML solutions architect on the global life sciences team. In Part 1, we looked at various options to ingest and store sensitive healthcare data using AWS. The post described our shared responsibility model and provided a […]

Read More

https://aws.amazon.com/fr/blogs/architecture/category/analytics/

# AWS – Events

AWSOME DAYS

https://aws.amazon.com/fr/events/awsomeday/

aws SUMMIT PARIS

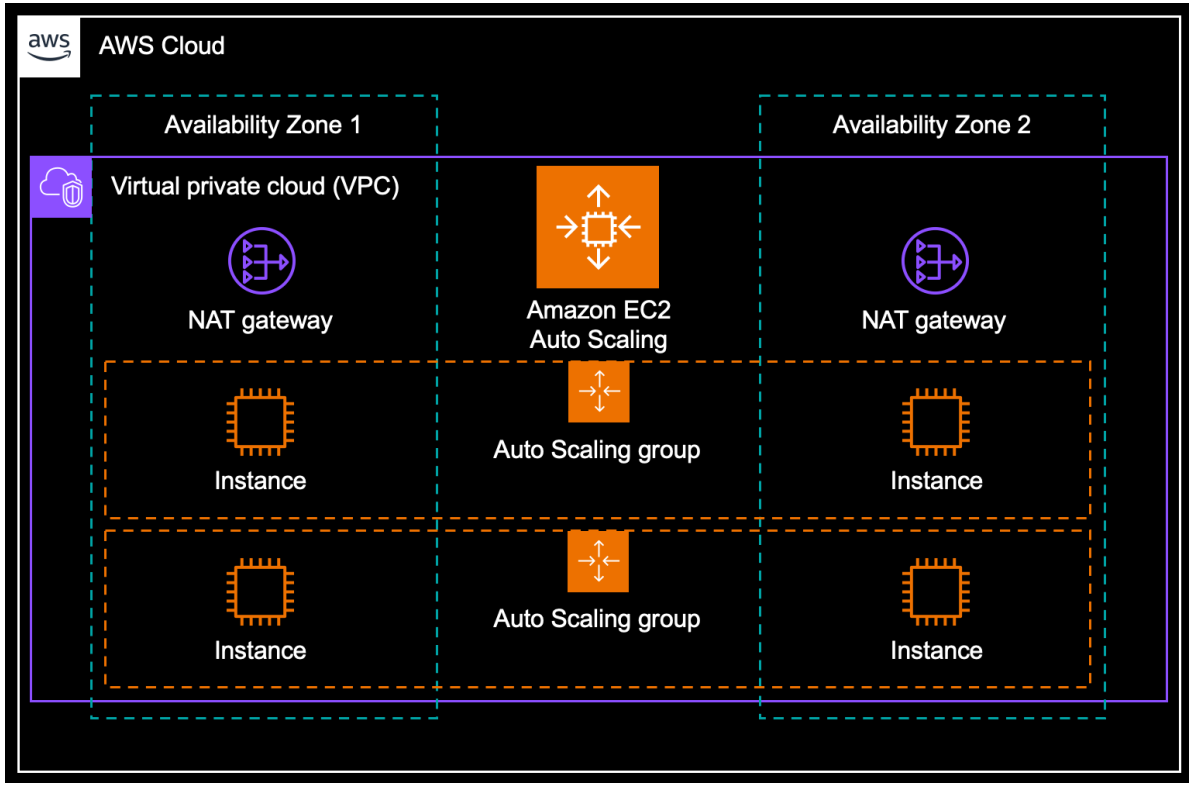
https://aws.amazon.com/fr/events/summits/emea/paris/




https://aws.amazon.com/fr/developer/community/usergroups/

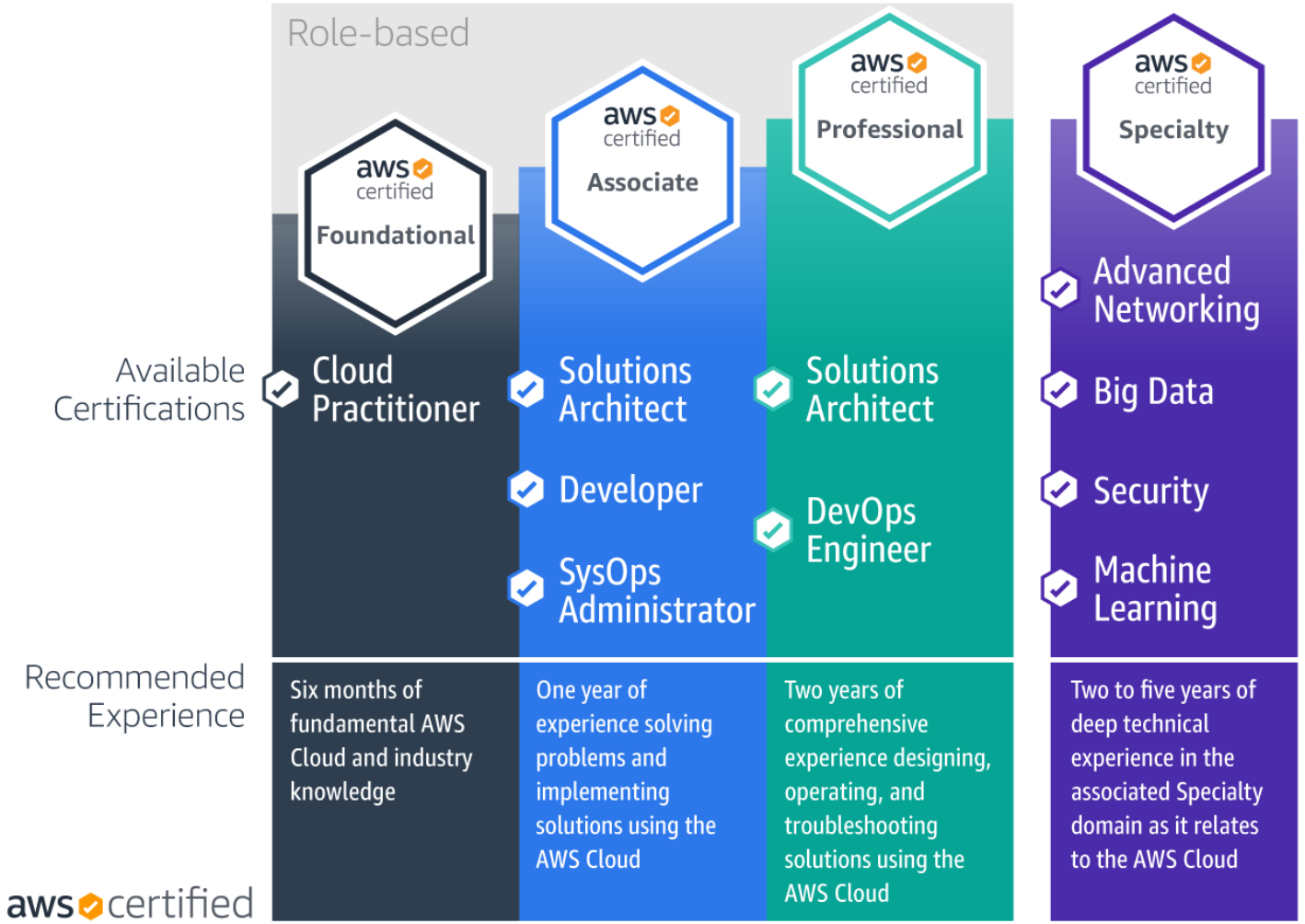
Bordeaux
Côte d'Azur
Grenoble
Lille
Lyon
Montpellier
Nantes
Paris
Rennes
Rouen
Toulouse

# AWS Architecture Icons




https://aws.amazon.com/architecture/icons/?nc1=h_ls

# AWS - certifications



# Azure

## Basics concepts



# What is Azure?

Azure is Microsoft's cloud computing platform. Azure is a continually expanding set of cloud services that help your organization meet your current and future business challenges. Azure gives you the freedom to build, manage, and deploy applications on a massive global network using your favorite tools and frameworks.

Azure provides over 100 services that enable you to do everything from running your existing applications on virtual machines to exploring new software paradigms such as intelligent bots and mixed reality.

Many teams start exploring the cloud by moving their existing applications to virtual machines that run in Azure. While migrating your existing apps to virtual machines is a good start, the cloud is more than just "a different place to run your virtual machines".

For example, Azure provides AI and machine-learning services that can naturally communicate with your users through vision, hearing, and speech. It also provides storage solutions that dynamically grow to accommodate massive amounts of data. Azure services enable solutions that are simply not feasible without the power of the cloud.

# Azure services

## Deploy your first solution in 10 minutes or less

Try out these short tutorials on how to use Azure and start building projects right away.



### Launch a Linux virtual machine

Deploy a Linux virtual machine using CLI.



### Launch a Windows virtual machine

Create a Windows virtual machine with PowerShell.



### Build a web app

Deploy a sample .NET, Node.js, Java, PHP, Python or Ruby app.



### Store and transfer data and apps

Access blob, table, and queue storage.



### Build a serverless app

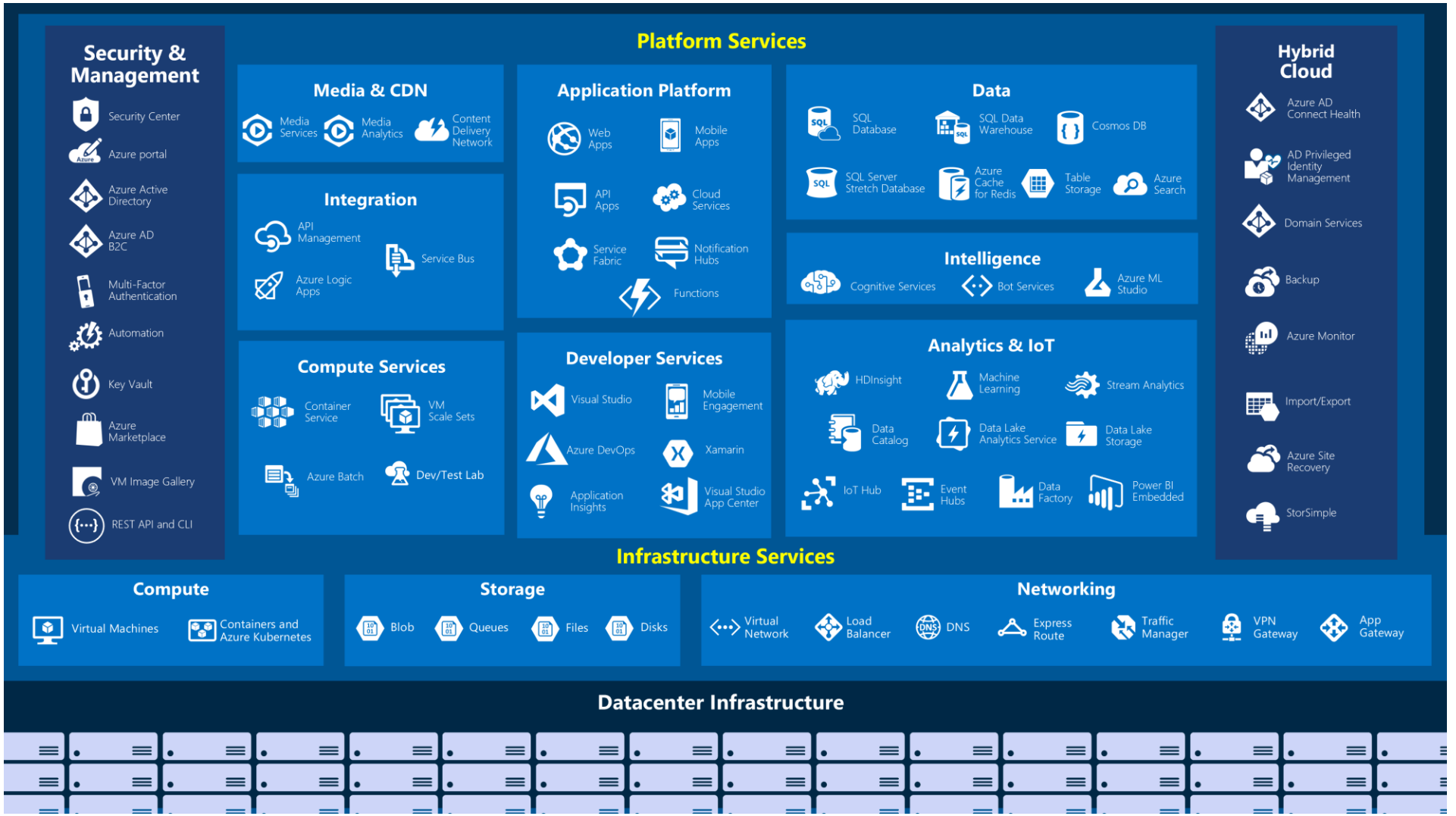
Create a "hello world" function in the Azure portal.




### Build a data-driven app

Create an Azure SQL database in the Azure portal.

# Azure services



# Azure charts




https://azurecharts.com/

# Compute

| Virtual Machines | Linux Virtual Machines | Azure Kubernetes Service (AKS) |
| --- | --- | --- |
|  |  |  |
| Provision Windows and Linux virtual machines in seconds | Provision virtual machines for Ubuntu, Red Hat, and more | Simplify the deployment, management, and operations of Kubernetes |

| Azure Functions |
| --- |
|  |
| Process events with serverless code |

# Databases

| Azure SQL Database | Azure Cosmos DB | Table Storage |
| --- | --- | --- |
|  Managed relational SQL Database as a service |  Globally distributed, multi-model database for any scale |  NoSQL key-value store using semi-structured datasets |

| Azure Database for PostGre | Azure Database Migration Service |
| --- | --- |
|  Managed MySQL database service for app developers |  Simplify on-premises database migration to the cloud |

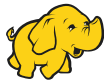
# Networking

| Azure Firewall | Azure DDoS Protection | VPN Gateway |
| --- | --- | --- |
|  Native firewalling capabilities with built-in high availability, unrestricted cloud scalability, and zero maintenance |  Protect your applications from Distributed Denial of Service (DDoS) attacks |  Establish secure, cross-premises connectivity |

| Azure DNS | ExpressRoute |
| --- | --- |
|  Host your DNS domain in Azure |  Dedicated private network fiber connections to Azure |

# Analytics

| Data Factory | SQL Data Warehouse | Azure Analysis Services |
|---|---|---|
|  |  |  |
| Hybrid data integration at enterprise scale, made easy | Elastic data warehouse as a service with enterprise-class features | Enterprise-grade analytics engine as a service |

| Azure Stream Analytics | Azure Databricks | Azure HDInsigth |
|---|---|---|
|  |  |  |
| Real-time data stream processing from millions of IoT devices | Fast, easy, and collaborative Apache Spark-based analytics platform | HDInsight supports the latest open source projects from the Apache Hadoop and Spark ecosystems. |

# Storage

| Blob Storage | Azure Backup | Azure Data Lake Storage |
| --- | --- | --- |
|  REST-based object storage for unstructured data |  Simplify data protection and protect against ransomware |  Massively scalable, secure data lake functionality built on Azure Blob Storage |

# Internet of Things

| Azure IoT Hub | Event Hubs | Azure Time Series Insights |
| --- | --- | --- |
|  |  |  |
| Connect, monitor and manage billions of IoT assets | Receive telemetry from millions of devices | Explore and analyze time-series data from IoT devices |

# Web

| Logic Apps | Web Apps | Mobile Apps |
| --- | --- | --- |
|  |  |  |
| Automate business processes | Quickly create and deploy mission critical web apps at scale | Build mobile apps for any device |

# Azure

## Practice



# Azure Portal



# Azure account

## Qu'est-ce que j'obtiens ?

Avec votre compte gratuit Azure, vous bénéficiez de tous ces éléments et vous n'êtes facturé que si vous effectuez une mise à niveau.

**12 mois**

de services gratuits populaires

\+

**Crédit de 170 €**

pour découvrir tout service Azure pendant 30 jours

\+

**Toujours gratuits**

Plus de 25 services

## Que dois-je savoir d'autre ?

**Dois-je payer quelque chose pour le compte gratuit Azure ?**

Non. Vous démarrez gratuitement et vous bénéficiez d'un crédit de 170 € que vous pouvez utiliser dans les 30 jours. Vous n'est pas facturé, même si vous commencez à utiliser des services, tant que vous n'avez pas procédé à une mise à niveau.

**Que se passe-t-il un fois que j'ai utilisé mon crédit de 170 € ou que la période de 30 jours est terminée ?**

Vous pouvez effectuer une mise à niveau vers un abonnement assorti d'un paiement à l'utilisation pour continuer à accéder aux produits gratuits : certains sont gratuits pendant les 12 premiers mois uniquement, d'autres le sont pour toujours.

**Que se passe-t-il à la fin des 12 mois de gratuité des produits ?**

Pendant les 12 mois qui suivent votre mise à niveau vers un abonnement assorti d'un paiement à l'utilisation, certains produits populaires sont gratuits. À la fin de ces 12 mois, vous serez facturé pour ces produits aux prix du paiement à l'utilisation.

**De quoi ai-je besoin pour m'inscrire ?**

Un numéro de téléphone, une carte bancaire et un nom d'utilisateur de compte Microsoft. Nous utilisons ces informations pour vérifier votre identité. Votre carte de crédit n'est pas débitée, mais il est possible qu'une retenue de vérification d'un dollar apparaisse sur votre compte. Elle sera supprimée dans les trois à cinq jours.

# LAB – Manage services with the Azure portal



Your turn !

**This module is part of these learning paths**
Azure fundamentals

- Introduction — 3 min ✓
- Azure management options — 7 min ✓
- Navigate the portal — 10 min ✓
- Exercise - Work with panes — 10 min ✓
- Exercise - Use the Azure portal — 10 min ✓
- Azure Portal dashboards — 15 min ✓
- Exercise - Customize the dashboard — 10 min ✓
- Access public and private preview features — 5 min ✓
- Summary — 3 min ✓

https://docs.microsoft.com/en-us/learn/modules/tour-azure-portal/

# Azure fundamentals



Your turn !

2400 XP

## Microsoft Azure Fundamentals: Describe cloud concepts

52 min • Learning Path • 3 Modules

https://learn.microsoft.com/en-us/training/paths/microsoft-azure-fundamentals-describe-cloud-concepts/

4700 XP

## Microsoft Azure Fundamentals: Describe Azure architecture and services

3 hr 25 min • Learning Path • 4 Modules

https://learn.microsoft.com/en-us/training/paths/azure-fundamentals-describe-azure-architecture-services/

3300 XP

## Microsoft Azure Fundamentals: Describe Azure management and governance

1 hr 52 min • Learning Path • 4 Modules

https://learn.microsoft.com/en-us/training/paths/describe-azure-management-governance/

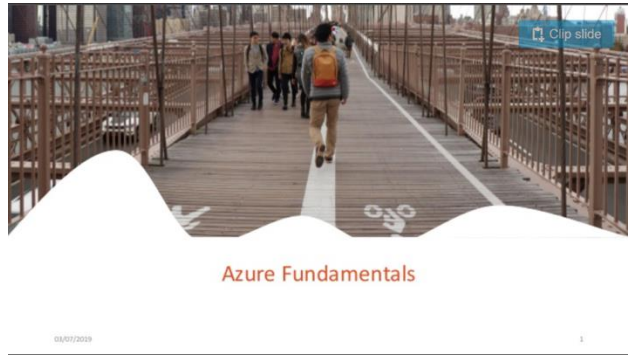
6200 XP

## Manage identity and access

5 hr 11 min • Learning Path • 4 Modules

Intermediate | Security Engineer | Azure | Microsoft Entra Verified ID | Microsoft Entra ID

https://learn.microsoft.com/en-us/training/paths/manage-identity-access-new/




https://www.slideshare.net/AlexandreBERGERE/azure-fundamentals-153339148



# Azure fundamentals



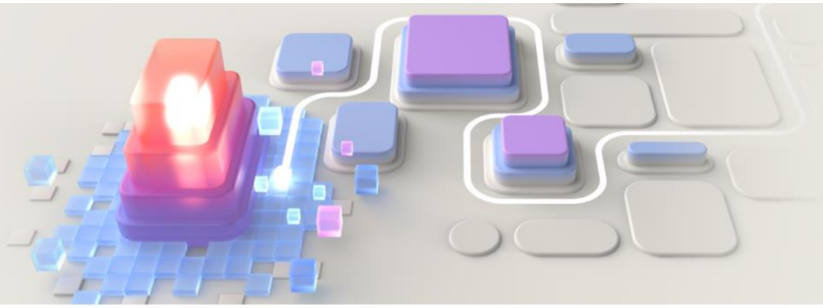
Your turn !

COURSE

## Microsoft Azure Fundamentals

Course AZ-900T00-A: Microsoft Azure Fundamentals

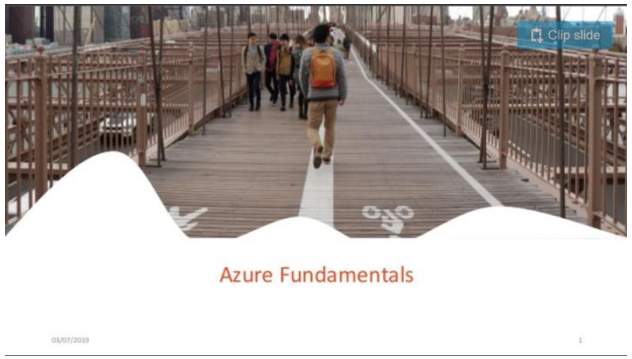
Get started with self-directed learning | Search for a training provider



### At a glance

- **Level**: Beginner
- **Product**: Azure
- **Role**: Administrator
- **Languages**: English, Arabic, Chinese (Simplified), Chinese (Traditional), French, German, Indonesian, Italian, Japanese, Korean, Portuguese (Brazil), Russian, Spanish
- **Course Duration**: 1 day
- **Related Certifications**: Microsoft Certified: Azure Fundamentals

https://learn.microsoft.com/fr-fr/training/courses/az-900t00




https://www.slideshare.net/AlexandreBERGERE/azure-fundamentals-153339148

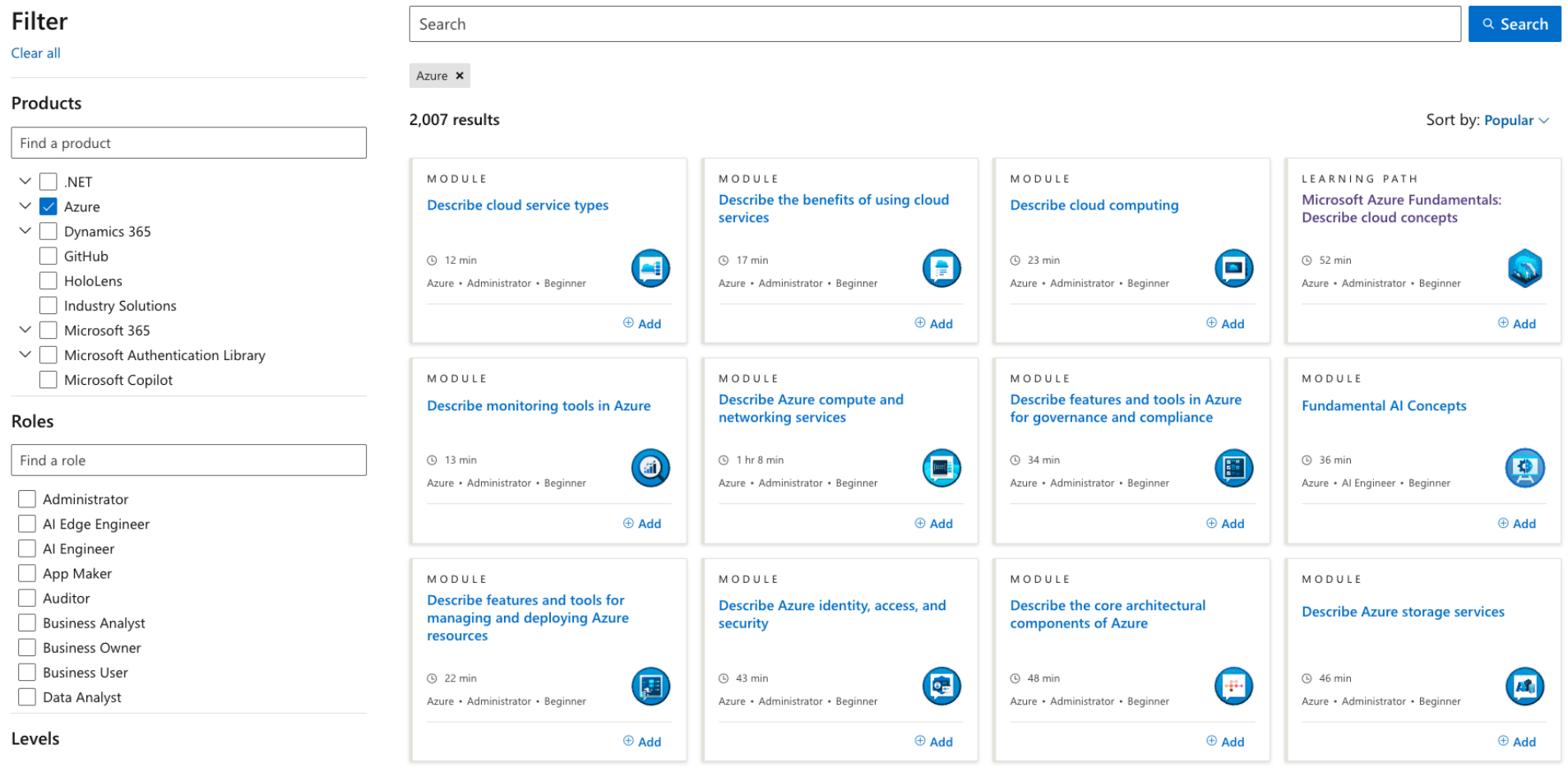


# Azure

## Documentation



# Azure - training




https://learn.microsoft.com/fr-fr/training/browse/?products=azure

# Azure – cloud workshop












https://microsoftcloudworkshop.com/

# Azure – certifications

## Find an Azure certification exam

**AZ-100**
Microsoft Azure Infrastructure and Deployment
(retires May 1, 2019)

**AZ-101**
Microsoft Azure Integration and Security
(retires May 1, 2019)

**AZ-102**
Microsoft Azure Administrator Certification Transition
(retires June 30, 2019)

**AZ-203**
Developing Solutions for Microsoft Azure

**AZ-300**
Microsoft Azure Architect Technologies

**AZ-301**
Microsoft Azure Architect Design

**AZ-302**
Microsoft Azure Solutions Architect Certification Transition
(released September 24, 2018; retires June 30, 2019)

**AZ-400**
Microsoft Azure DevOps Solutions

BETA **AZ-500**
Microsoft Azure Security Technologies
(released April 1, 2019)

**AZ-900**
Microsoft Azure Fundamentals
(released December 31, 2018)

**70-473**
Designing and Implementing Cloud Data Platform Solutions

**70-475**
Designing and Implementing Big Data Analytics Solutions

**70-487**
Developing Microsoft Azure and Web Services

**70-537**
Configuring and Operating a Hybrid Cloud with Microsoft Azure Stack

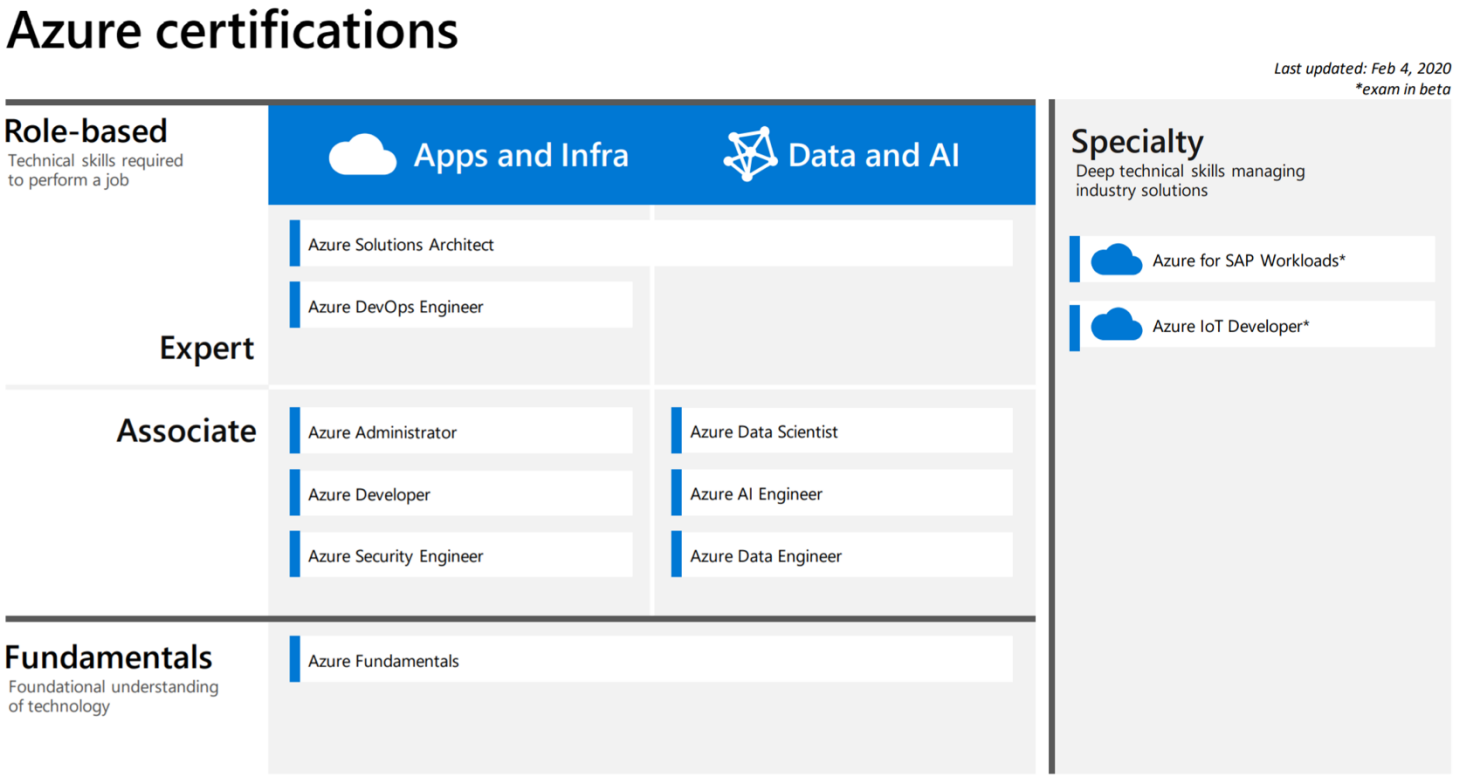
**70-774**
Perform Cloud Data Science with Azure Machine Learning
(retires June 30, 2019)

**70-775**
Perform Data Engineering on Microsoft HDInsight
(retires June 30, 2019)

**70-776**
Engineering Data with Microsoft Cloud Service
(retires June 30, 2019)

https://www.microsoft.com/en-us/learning/azure-exams.aspx

# Microsoft training and certifications



# Awesome Azure Learning

## Awesome Azure Learning 


This repo includes numerous references for Azure learning, especially for the Azure Certs, Azure Architecture, and any other learning materials e.g. Security topics.



https://github.com/ddneves/awesome-azure-learning